EECE 2140: Computing Fundamentals for Engineers

Fall 2026

Assignment #02: Binary Tree Interview Questions

Student Name	Catalina Buitano
NUID	<Your NUID>
Email	<your.email@northeastern.edu>
Instructor	Dr. Fatema Nafa
Due Date	<Enter Due Date>

Problem 1:

Description: Tree depth shows up in balancing checks, recursion limits and anywhere you need to know how tall a structure has grown. The depth tells you the longest chain of decisions from the top of the tree to the bottom.

Question: Return the maximum depth of the tree: the number of edges on the longest path from the root to a leaf. An empty tree has depth -1.

Goal: Return the number of edges on the longest path from the root down to a leaf. An empty tree has depth -1.

Inputs: The root of a binary tree. Here it is the tree built from arr = {5, 12, 8, 21, 3, 17, 9}.

Outputs: One integer, the number of edges on the longest root-to-leaf path. Expected output: 2.

Constraints: The depth is counted in edges, not nodes, so a single node has depth 0 and an empty tree has depth -1. Every node has at most two children.

Assumptions: The tree is not ordered in any way, so both subtrees always have to be checked. The values stored in the nodes do not matter, only the shape.

Edge Cases:

1. Empty tree (root = NULL), which should return -1
2. A single node, which should return 0
3. A chain where every node has one child, which is the deepest tree possible for n nodes
4. A full, balanced tree like the reference tree

Designing Solution: When I looked at the tree by hand, the depth of the whole tree is just one more than the deeper of its two subtrees, because going from the root to either child adds one edge. That gave me a recursion: ask the left subtree for its depth, ask the right subtree for its depth, take the bigger one and add 1. The base case is the empty tree returning -1, which makes a leaf come out as $1 + \max(-1, -1) = 0$, which is correct since a leaf has no edges below it.

Pseudocode Problem 1: Maximum Depth of a Binary Tree

Algorithm 1 MAXDEPTH

Input: node, the root of a tree or subtree

Output: the number of edges on the longest root-to-leaf path (-1 if empty)

Initial call: MAXDEPTH(root)

```
1: procedure MAXDEPTH(node)
2:     if node = NULL then
3:         return −1
4:     end if
5:     leftDepth ← MAXDEPTH(node.left)
6:     rightDepth ← MAXDEPTH(node.right)
7:     return 1 + max(leftDepth, rightDepth)
8: end procedure
```

Explanation: The recursion is correct because the longest path from a node down to a leaf has to go through one of its two children, so it is one edge longer than the longer of the two child paths. Line 7 takes exactly that maximum and adds the one edge. The base case returns -1 for NULL, so a leaf correctly gets $1 + \max(-1, -1) = 0$, and an empty tree returns -1 as the question asks. Every call is made on a smaller subtree, so the recursion always reaches the NULL base case and stops. A chain is handled the same way as any other tree, it just makes the recursion deeper.

Worked Example:

Every call in the order it happens (N means a NULL child):

Call #	Call	Lines executed	Returns
1	MAXDEPTH(5)	2, 5, 6, 7	$1 + \max(1, 1) = 2$
2	MAXDEPTH(12)	2, 5, 6, 7	$1 + \max(0, 0) = 1$
3	MAXDEPTH(21)	2, 5, 6, 7	$1 + \max(-1, -1) = 0$
4	MAXDEPTH(N) left of 21	2, 3	-1
5	MAXDEPTH(N) right of 21	2, 3	-1
6	MAXDEPTH(3)	2, 5, 6, 7	0
7	MAXDEPTH(N) left of 3	2, 3	-1
8	MAXDEPTH(N) right of 3	2, 3	-1
9	MAXDEPTH(8)	2, 5, 6, 7	$1 + \max(0, 0) = 1$
10	MAXDEPTH(17)	2, 5, 6, 7	0
11	MAXDEPTH(N) left of 17	2, 3	-1
12	MAXDEPTH(N) right of 17	2, 3	-1
13	MAXDEPTH(9)	2, 5, 6, 7	0
14	MAXDEPTH(N) left of 9	2, 3	-1
15	MAXDEPTH(N) right of 9	2, 3	-1

Final output: 2. This matches the expected output of 2.

Frequency Count:

Line	Operation	Cost	Frequency	Count for the example
2	test node = NULL	1	$2n + 1$	15
3	return -1	1	$n + 1$	8
5	call MAXDEPTH(node.left)	1	n	7
6	call MAXDEPTH(node.right)	1	n	7
7	return $1 + \max(\dots)$	1	n	7
	Total T(n)		$6n + 2$	44

Where the frequencies come from: the procedure is called once for every real node (n calls) and once for every NULL pointer ($n + 1$ calls), so line 2 runs $2n + 1$ times. Only the NULL calls reach line 3, and only the real-node calls reach lines 5 to 7.

Adding the rows:

$T(n) = (2n + 1) + (n + 1) + n + n + n$ *(add every row's frequency)*

$T(n) = 6n + 2$ *(collect the n terms and the constants)*

$T(7) = 6(7) + 2 = 44$ *(plug in n = 7)*

Time and Space Complexity: The dominant term is 6n, so the running time is $\Theta(n)$ in both the best and worst case, since every node and every NULL pointer gets visited exactly once no matter what the tree looks like. The extra space is the recursion stack, which is $\Theta(h)$: $\Theta(\log n)$ for a balanced tree in the best case and $\Theta(n)$ for a chain in the worst case. The worst-case input is a chain where every node has only one child, because it makes the stack as deep as possible.

Twist: The frequency count does not change. A chain of 7 nodes still has $n = 7$ real nodes and $n + 1 = 8$ NULL pointers, so every row in the table has the same frequency and $T(7) = 6(7) + 2 = 44$ again. The only differences are that the answer becomes 6 instead of 2, and the recursion stack gets 8 calls deep instead of 4, so the time stays the same but the space gets worse.

C++ Implementation:

```
// Problem 1
int maxDepth(Node* node) {
    if (node == nullptr) return -1;
    int leftDepth = maxDepth(node->left);
    int rightDepth = maxDepth(node->right);
    return 1 + max(leftDepth, rightDepth);
}
void runProblem1() {
    Node* root = readTree();
    cout << "Maximum depth: " << maxDepth(root) << endl;
}
```

Test Cases:

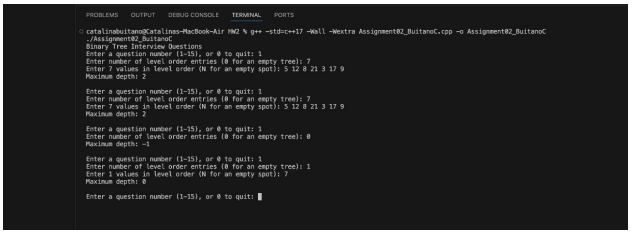


Figure 1: Test Cases 1-4

Test 1 (given example)
Input: the tree from arr = {5, 12, 8, 21, 3, 17, 9}
Expected Output: 2

Test 2 (edge case: empty tree)
Input: root = NULL
Expected Output: −1

Test 3 (edge case: single node)
Input: {7}

Expected Output: 0

Test 4 (edge case: chain of 7 nodes)
Input: $1 \to 2 \to 3 \to 4 \to 5 \to 6 \to 7$ (all left children)
Expected Output: 6

Problem 2:

Description: Adding up every value in a tree is the simplest kind of aggregation, the same pattern used to total costs in an org chart or sizes in a folder tree. It is a good way to compare the recursive traversal with the queue-based level order traversal.

Question: Return the sum of the values stored in all nodes of the tree. Write it once recursively and once with a queue (level order), and compare the two frequency counts and the extra space each one needs.

Goal: Return the sum of every value stored in the tree. Expected output: 75.

Inputs: The root of a binary tree. Here it is the tree built from arr = {5, 12, 8, 21, 3, 17, 9}.

Outputs: One integer, the sum of all node values. Expected output: 75.

Constraints: Both versions must return the same total. The queue version must visit the nodes in level order.

Assumptions: Values can be any integers, including negative numbers, and the sum fits in a normal integer. An empty tree has a sum of 0.

Edge Cases:

1. Empty tree, which should return 0
2. A single node, which should return its own value
3. Negative values, which must still be added correctly
4. A chain (worst case for the recursion stack) and a complete tree (worst case for the queue)

Designing Solution: The sum of a tree is the root's value plus the sum of the left subtree plus the sum of the right subtree, and an empty tree adds 0. That is the same shape as Question 1, so the recursive version came out almost the same. For the queue version I used level order: start with the root in a queue, and every time I take a node out I add its value to a running total and put its children in. Both versions touch every node once, so I expected both to be $\Theta(n)$, and the real difference is how much extra memory each one needs.

Pseudocode Problem 2: Sum of All Nodes

Algorithm 2a SUM (recursive)

Input: node, the root of a tree or subtree

Output: the sum of all values in that tree (0 if empty)

Initial call: SUM(root)

```
1:  procedure SUM(node)
2:      if node = NULL then
3:          return 0
4:      end if
5:      leftSum ← SUM(node.left)
6:      rightSum ← SUM(node.right)
7:      return node.value + leftSum + rightSum
8:  end procedure
```

Algorithm 2b SUMLEVEL (queue, level order)

Input: root, the root of the tree

Output: the sum of all values in the tree (0 if empty)

Initial call: SUMLEVEL(root)

```
 1:  procedure SUMLEVEL(root)
 2:      if root = NULL then
 3:          return 0
 4:      end if
 5:      total ← 0
 6:      Q ← empty queue
 7:      ENQUEUE(Q, root)
 8:      while Q is not empty do
 9:          node ← DEQUEUE(Q)
10:          total ← total + node.value
11:          if node.left ≠ NULL then
12:              ENQUEUE(Q, node.left)
13:          end if
14:          if node.right ≠ NULL then
15:              ENQUEUE(Q, node.right)
16:          end if
17:      end while
18:      return total
19:  end procedure
```

Explanation: The recursive version is correct because the sum of a tree splits exactly into the root's value, the sum of the left subtree and the sum of the right subtree, and the empty tree adds 0, so missing children do not change the answer. The queue version is correct because every node goes into the queue exactly once, when its parent is dequeued (the root goes in on line 7), and every node that comes out of the queue is added to total on line 10. So every value is added exactly once. Both versions return 0 for an empty tree, and negative values work because the code only adds.

Worked Example:

Recursive version (N means a NULL child; each NULL call runs lines 2, 3 and returns 0):

Call #	Call	Lines executed	Returns
1	SUM(5)	2, 5, 6, 7	5 + 36 + 34 = 75
2	SUM(12)	2, 5, 6, 7	12 + 21 + 3 = 36
3	SUM(21)	2, 5, 6, 7	21 + 0 + 0 = 21

Call #	Call	Lines executed	Returns
4, 5	SUM(N), SUM(N) under 21	2, 3	0, 0
6	SUM(3)	2, 5, 6, 7	3 + 0 + 0 = 3
7, 8	SUM(N), SUM(N) under 3	2, 3	0, 0
9	SUM(8)	2, 5, 6, 7	8 + 17 + 9 = 34
10	SUM(17)	2, 5, 6, 7	17 + 0 + 0 = 17
11, 12	SUM(N), SUM(N) under 17	2, 3	0, 0
13	SUM(9)	2, 5, 6, 7	9 + 0 + 0 = 9
14, 15	SUM(N), SUM(N) under 9	2, 3	0, 0

Queue version, one row per pass of the while loop (queue shown front to back after the pass):

Pass	Dequeued	total	Enqueued	Queue after the pass
start	—	0	5	[5]
1	5	0 + 5 = 5	12, 8	[12, 8]
2	12	5 + 12 = 17	21, 3	[8, 21, 3]
3	8	17 + 8 = 25	17, 9	[21, 3, 17, 9]
4	21	25 + 21 = 46	—	[3, 17, 9]
5	3	46 + 3 = 49	—	[17, 9]
6	17	49 + 17 = 66	—	[9]
7	9	66 + 9 = 75	—	[]

Final output: 75 from both versions, which matches the expected output.

Frequency Count:

Recursive version:

Line	Operation	Cost	Frequency	Count for the example
2	test node = NULL	1	2n + 1	15
3	return 0	1	n + 1	8
5	call SUM(node.left)	1	n	7
6	call SUM(node.right)	1	n	7
7	return node.value + leftSum + rightSum	1	n	7
	Total T(n)		6n + 2	44

$T(n) = (2n + 1) + (n + 1) + n + n + n = 6n + 2$ *(add the rows and collect terms)*

$T(7) = 6(7) + 2 = 44$ *(plug in n = 7; the trace has 7 calls × 4 lines + 8 calls × 2 lines = 28 + 16 = 44)*

Queue version:

Line	Operation	Cost	Frequency	Count for the example
2	test root = NULL	1	1	1
3	return 0	1	0	0
5	total ← 0	1	1	1
6	Q ← empty queue	1	1	1
7	ENQUEUE(Q, root)	1	1	1
8	test Q is not empty	1	n + 1	8
9	DEQUEUE	1	n	7

Line	Operation	Cost	Frequency	Count for the example
10	add to total	1	n	7
11	test node.left $\neq$ NULL	1	n	7
12	ENQUEUE left child	1	L (left children)	3
14	test node.right $\neq$ NULL	1	n	7
15	ENQUEUE right child	1	R (right children)	3
18	return total	1	1	1
	Total T(n)		6n + 5	47

Every node except the root is somebody's child, so L + R = n − 1. The while test runs once per node plus one last time when the queue is empty, which is n + 1.

$T(n) = 1 + 1 + 1 + 1 + (n + 1) + n + n + n + n + (L + R) + 1$ *(add the rows; line 3 is 0 for a non-empty tree)*

$T(n) = 5 + (n + 1) + 4n + (n - 1)$ *(group the constants and replace L + R with n − 1)*

$T(n) = 6n + 5$ *(collect terms)*

$T(7) = 6(7) + 5 = 47$ *(plug in n = 7; matches the table: 1+1+1+1+8+7+7+7+3+7+3+1 = 47)*

Time and Space Complexity: Both versions are $\Theta(n)$ in the best and worst case, since every node is visited exactly once; the queue version only costs 3 more steps (6n + 5 vs 6n + 2) because of its setup lines. The extra space is different. The recursive version uses the call stack, $\Theta(h)$, which is $\Theta(\log n)$ for a balanced tree and $\Theta(n)$ for a chain. The queue version holds at most one level at a time, so its extra space is $\Theta(w)$ where w is the widest level: a chain only needs $\Theta(1)$, but a complete tree has about n/2 nodes on its last level, so it needs $\Theta(n)$. For our tree the queue peaks at 4 nodes (pass 3) and the stack peaks at 4 calls deep, so they tie here.

Twist: The frequency counts are almost the same: 6n + 2 for recursion and 6n + 5 for the queue, so both are $\Theta(n)$. The bigger difference is space, and the two versions have opposite worst cases. Recursion is worst on a tall, thin tree (a chain makes the stack $\Theta(n)$), and the queue is worst on a short, wide tree (a complete tree makes the queue $\Theta(n)$).

C++ Implementation:

```
// Problem 2
int sumRecursive(Node* node) {
    if (node == nullptr) return 0;
    int leftSum = sumRecursive(node->left);
    int rightSum = sumRecursive(node->right);
    return node->value + leftSum + rightSum;
}

int sumLevel(Node* root) {
    if (root == nullptr) return 0;
    int total = 0;
    queue<Node*> q;
    q.push(root);
    while (!q.empty()) {
        Node* node = q.front(); q.pop();
        total += node->value;
        if (node->left != nullptr) q.push(node->left);
        if (node->right != nullptr) q.push(node->right);
    }
```

```
    return total;
}
void runProblem2() {
    Node* root = readTree();
    cout << "Sum (recursive): " << sumRecursive(root) << endl;
    cout << "Sum (queue, level order): " << sumLevel(root) << endl;
}
```

Test Cases:

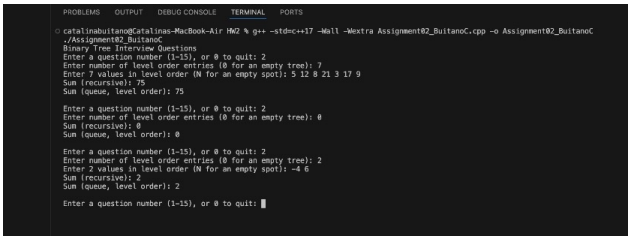


Figure 2: Test Cases 1-3

Test 1 (given example)
Input: the tree from arr
Expected Output: 75 from both versions

Test 2 (edge case: empty tree)
Input: root = NULL
Expected Output: 0

Test 3 (edge case: negative values)
Input: $\{-4, 6\}$
Expected Output: 2

Problem 3:

Description: Counting nodes with exactly one child is a quick way to measure how lopsided or incomplete a tree is. In a tree built from an array, it also shows where the last level stops.

Question: Return the number of nodes that have a left child or a right child, but not both. Use the tree from arr after INSERT(root, 4) with the level order rules from the lecture.

Goal: Return how many nodes have a left child or a right child, but not both. Expected output: 1 (node 21, whose only child is 4).

Inputs: The root of a binary tree. Here it is the tree from arr after INSERT(root, 4), so n = 8.

Outputs: One integer, the number of nodes with exactly one child. Expected output: 1 (node 21, whose only child is 4).

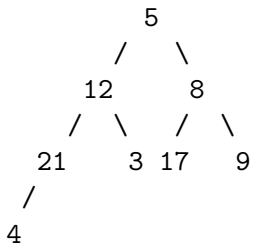
Constraints: A node counts only if exactly one of its two child pointers is NULL. Leaves (no children) and full nodes (two children) do not count.

Assumptions: INSERT puts the new node in the first open spot in level order, which is the left child of node 21. The tree is not ordered by value.

Edge Cases:

1. Empty tree, which should return 0
2. The tree from arr before the insert (n = 7, odd), which should return 0
3. A chain of n nodes, where every node except the leaf has one child, so the answer is n − 1
4. A single node, which is a leaf and should return 0

First I applied INSERT(root, 4) with the level order rule from the lecture, which puts the new node in the first open spot when scanning level by level. Positions 0 to 6 are full, so 4 goes to position 7, which is the left child of position 3 (node 21). The tree now has n = 8 nodes:



Designing Solution: I visit every node and check whether exactly one of its two children is NULL. The answer for a tree is the count from the left subtree, plus the count from the right subtree, plus 1 if the current node itself has exactly one child. An empty tree contributes 0.

Pseudocode Problem 3: Count the Nodes That Have Exactly One Child

Algorithm 3 COUNTONE

Input: node, the root of a tree or subtree

Output: the number of nodes in that tree with exactly one child

Initial call: COUNTONE(root)

```
 1: procedure COUNTONE(node)
 2:     if node = NULL then
 3:         return 0
 4:     end if
 5:     leftCount ← COUNTONE(node.left)
 6:     rightCount ← COUNTONE(node.right)
 7:     if (node.left = NULL) ≠ (node.right = NULL) then
 8:         return leftCount + rightCount + 1
 9:     end if
10:     return leftCount + rightCount
11: end procedure
```

Explanation: Line 7 checks whether node.left is NULL and node.right is NULL give different answers, which happens exactly when one child is present and the other is missing. If both are NULL (a leaf) or both are present, the two tests give the same answer and the node is not counted. Every node is visited once, so every one-child node is counted exactly once and added to the counts coming up from its subtrees. An empty tree returns 0 on line 3.

Worked Example:

Call #	Call	Lines executed	Returns
1	COUNTONE(5)	2, 5, 6, 7, 10	$1 + 0 = 1$
2	COUNTONE(12)	2, 5, 6, 7, 10	$1 + 0 = 1$
3	COUNTONE(21)	2, 5, 6, 7, 8	$0 + 0 + 1 = 1$ (left is 4, right is NULL)
4	COUNTONE(4)	2, 5, 6, 7, 10	$0 + 0 = 0$
5, 6	COUNTONE(N), COUNTONE(N) under 4	2, 3	0, 0
7	COUNTONE(N) right of 21	2, 3	0
8	COUNTONE(3)	2, 5, 6, 7, 10	0
9, 10	COUNTONE(N), COUNTONE(N) under 3	2, 3	0, 0
11	COUNTONE(8)	2, 5, 6, 7, 10	$0 + 0 = 0$
12	COUNTONE(17)	2, 5, 6, 7, 10	0
13, 14	COUNTONE(N), COUNTONE(N) under 17	2, 3	0, 0
15	COUNTONE(9)	2, 5, 6, 7, 10	0
16, 17	COUNTONE(N), COUNTONE(N) under 9	2, 3	0, 0

Final output: 1. This matches the expected output (node 21).

Frequency Count:

Here $n = 8$, and c is the number of one-child nodes ($c = 1$ in the example).

Line	Operation	Cost	Frequency	Count for the example
2	test node = NULL	1	$2n + 1$	17
3	return 0	1	$n + 1$	9
5	call COUNTONE(node.left)	1	n	8
6	call COUNTONE(node.right)	1	n	8
7	test exactly one child	1	n	8
8	return count + 1	1	c	1
10	return count	1	$n - c$	7
	Total T(n)		$7n + 2$	58

$T(n) = (2n + 1) + (n + 1) + n + n + n + c + (n - c)$ *(add every row)*

$T(n) = 7n + 2$ *(the c and $-c$ cancel, since every real node returns from exactly one of lines 8 or 10)*

$T(8) = 7(8) + 2 = 58$ *(plug in $n = 8$)*

Check with the trace: 8 real-node calls $\times$ 5 lines = 40, and 9 NULL calls $\times$ 2 lines = 18, so $40 + 18 = 58$.

Time and Space Complexity: $T(n) = 7n + 2$, so the time is $\Theta(n)$ in the best and worst case, because every node has to be checked and the count does not depend on c. The extra space is the recursion stack, $\Theta(h)$: $\Theta(\log n)$ for a balanced tree and $\Theta(n)$ for a chain. A chain is the worst-case input, because it maximizes the stack depth (it is also the input with the most one-child nodes, $c = n - 1$, but that does not change the time).

Twist: In a tree built from an array, the node at index i has a left child only if $2i + 1 \leq n - 1$ and a right child only if $2i + 2 \leq n - 1$. A node has exactly one child when its left child exists but its right child does not:

$2i + 1 \leq n - 1$ and $2i + 2 > n - 1$ *(left index is in the array, right index is past the end)*

$2i + 1 \leq n - 1 < 2i + 2$ *(combine the two conditions)*

$2i + 1 = n - 1$ *(n − 1 is a whole number squeezed between two consecutive integers, so it must equal the lower one)*

$i = (n - 2) / 2$ *(solve for i)*

The left side $2i + 1$ is always odd. If n is even, then $n - 1$ is odd, so there is exactly one index $i = (n - 2)/2$ that works, which means exactly one node. If n is odd, then $n - 1$ is even, and an odd number can never equal an even number, so no node has exactly one child. A node can also never have only a right child, since index $2i + 2$ existing means $2i + 1$ exists too. For our example $n = 8$ is even, so $i = (8 - 2)/2 = 3$, which is node 21.

C++ Implementation:

```
// Problem 3
// INSERT from the lecture: put the new node in the first open spot in level order
Node* insertLevel(Node* root, int value) {
    Node* fresh = new Node(value);
    if (root == nullptr) return fresh;
    queue<Node*> q;
    q.push(root);
    while (!q.empty()) {
        Node* node = q.front(); q.pop();
        if (node->left == nullptr) { node->left = fresh; return root; }
        q.push(node->left);
        if (node->right == nullptr) { node->right = fresh; return root; }
        q.push(node->right);
    }
    return root;
}

int countOneChild(Node* node) {
    if (node == nullptr) return 0;
    int leftCount = countOneChild(node->left);
    int rightCount = countOneChild(node->right);
    if ((node->left == nullptr) != (node->right == nullptr)) {
        return leftCount + rightCount + 1;
    }
    return leftCount + rightCount;
}
void runProblem3() {
    Node* root = readTree();
    string token;
    cout << "Enter the value to INSERT (N to skip the insert): ";
    cin >> token;
    if (token != "N" && token != "n") root = insertLevel(root, stoi(token));
    cout << "Level order: "; printList(levelOrder(root)); cout << endl;
    cout << "Nodes with exactly one child: " << countOneChild(root) << endl;
}
```

Test Cases:

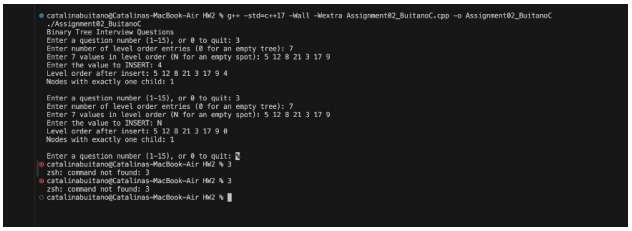
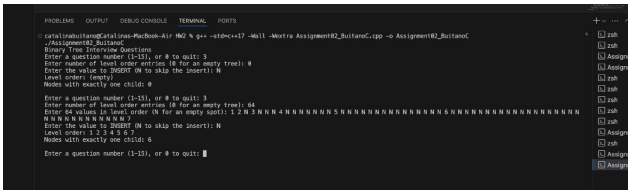


Figure 3: Test Cases 1 & 2


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
catalinabuitanog@Catalinas-MacBook-Air HW2 % g++ -std=c++17 -Wall -Wextra Assignment02_BuitanoC.cpp -o Assignment02_BuitanoC
./Assignment02_BuitanoC
Binary Tree Interview Questions
Enter a question number (1-15), or 0 to quit: 3
Enter number of level order entries (0 for an empty tree): 0
Enter the value to INSERT (N to skip the insert): N
Level order: (empty)
Nodes with exactly one child: 0
Enter a question number (1-15), or 0 to quit: 3
Enter number of level order entries (0 for an empty tree): 64
Enter 64 values in level order (N for an empty spot): 1 2 N 3 N N N 4 N N N N N N N 5 N N N N N N N N N N N N N N N 6 N N N N N N N N N N N N N N N

Constraints: Both the shape and the values must match. Two trees with the same values in a different shape are not identical.

Assumptions: Two empty trees count as identical. The trees can have different sizes, in which case the answer is false.

Edge Cases:

1. Both trees empty, which should return true
2. One tree empty and the other not, which should return false
3. Same values but a different shape (a node missing in one tree), which should return false
4. A difference only in the last node checked, which is the slowest false case

Designing Solution: Two trees are identical if their roots match and their left subtrees are identical and their right subtrees are identical. I walk both trees at the same time in preorder. There are three ways a pair of positions can turn out: both are NULL (fine, same shape), only one is NULL (different shape, return false), or both are real nodes, in which case the values have to match before I keep going. I return false as soon as I find a difference, so I don't waste time checking the right subtree if the left one already failed.

Pseudocode Problem 4: Are Two Trees Identical?

Algorithm 4 IDENTICAL

Input: a and b, the roots of two trees or subtrees

Output: true if the two trees have the same shape and values, otherwise false

Initial call: IDENTICAL(rootA, rootB)

```
 1: procedure IDENTICAL(a, b)
 2:     if a = NULL and b = NULL then
 3:         return true
 4:     end if
 5:     if a = NULL or b = NULL then
 6:         return false
 7:     end if
 8:     if a.value ≠ b.value then
 9:         return false
10:     end if
11:     if IDENTICAL(a.left, b.left) = false then
12:         return false
13:     end if
14:     return IDENTICAL(a.right, b.right)
15: end procedure
```

Explanation: Lines 2 to 7 cover every case where at least one side is NULL: both NULL means the shapes match at that spot, and only one NULL means the shapes differ. Once both nodes are real, line 8 compares the values, and lines 11 to 14 require both subtrees to be identical too. Two trees are identical exactly when the roots match and both pairs of subtrees are identical, so the recursion follows the definition directly. The algorithm stops at the first difference, which is why it is fast when the trees differ early.

Worked Example:

IDENTICAL(A, B), where every pair matches (N means a NULL pair):

Call #	Call	Lines executed	Returns
1	(5, 5)	2, 5, 8, 11, 14	true
2	(12, 12)	2, 5, 8, 11, 14	true
3	(21, 21)	2, 5, 8, 11, 14	true
4, 5	(N, N), (N, N) under 21	2, 3	true, true
6	(3, 3)	2, 5, 8, 11, 14	true
7, 8	(N, N), (N, N) under 3	2, 3	true, true
9	(8, 8)	2, 5, 8, 11, 14	true
10	(17, 17)	2, 5, 8, 11, 14	true
11, 12	(N, N), (N, N) under 17	2, 3	true, true
13	(9, 9)	2, 5, 8, 11, 14	true
14, 15	(N, N), (N, N) under 9	2, 3	true, true

IDENTICAL(A, C), where the last leaf is 9 in A but 10 in C:

Call #	Call	Lines executed	Returns
1	(5, 5)	2, 5, 8, 11, 14	false (passed up from (8, 8))
2	(12, 12)	2, 5, 8, 11, 14	true
3	(21, 21)	2, 5, 8, 11, 14	true
4, 5	(N, N), (N, N) under 21	2, 3	true, true
6	(3, 3)	2, 5, 8, 11, 14	true
7, 8	(N, N), (N, N) under 3	2, 3	true, true
9	(8, 8)	2, 5, 8, 11, 14	false (passed up from (9, 10))
10	(17, 17)	2, 5, 8, 11, 14	true
11, 12	(N, N), (N, N) under 17	2, 3	true, true
13	(9, 10)	2, 5, 8, 9	false

Final output: IDENTICAL(A, B) = true and IDENTICAL(A, C) = false. Both match the expected output.

Frequency Count:

The formula column is for the worst case, two identical trees of n nodes. The last column also shows the count for IDENTICAL(A, C).

Line	Operation	Cost	Frequency (identical trees)	Count (A, B)	Count (A, C)
2	test both NULL	1	2n + 1	15	13
3	return true	1	n + 1	8	6
5	test one NULL	1	n	7	7
6	return false	1	0	0	0
8	test values differ	1	n	7	7
9	return false	1	0	0	1
11	call left and test result	1	n	7	6
12	return false	1	0	0	0
14	call right and return	1	n	7	6
	Total T(n)		7n + 2	51	46

$T(n) = (2n + 1) + (n + 1) + n + n + n + n$ *(add the rows; the return-false lines never run for identical trees)*

$T(n) = 7n + 2$ *(collect terms)*

$T(7) = 7(7) + 2 = 51$ *(plug in n = 7; trace check: 7 pairs × 5 lines + 8 NULL pairs × 2 lines = 35 + 16 = 51)*

For (A, C): 6 matching pairs × 5 lines = 30, the (9, 10) pair runs 4 lines, and 6 NULL pairs × 2 lines = 12, so 30 + 4 + 12 = 46.

Time and Space Complexity: The dominant term is 7n, so the worst case is $\Theta(n)$. The worst case is two identical trees, because the algorithm can only answer true after it has compared every pair, including all n + 1 NULL pairs. The best case is $\Theta(1)$, when the roots already differ (for example 5 vs 6): only lines 2, 5, 8, 9 run, which is 4 steps. The extra space is the recursion stack, $\Theta(h)$.

Twist: It finds the difference at the last node in preorder, the bottom-right leaf (9 in A vs 10 in C), at call 13 on line 9. Since the preorder walk checks the root, then the whole left subtree, then the right subtree, the bottom-right leaf is the very last real pair it reaches. That is why the count for (A, C) is 46, which is almost the full 51: it only saved 5 steps: the two NULL pairs under 9 (4 steps), plus 1 step at the (9, 10) pair, which stopped after 4 lines instead of running 5. For two trees of n nodes, the best case is $\Theta(1)$ (the roots differ) and the worst case is $\Theta(n)$ (the trees are identical, or the only difference is in the last position checked, like here).

C++ Implementation:

```
// Problem 4
bool identical(Node* a, Node* b) {
    if (a == nullptr && b == nullptr) return true;
    if (a == nullptr || b == nullptr) return false;
    if (a->value != b->value) return false;
    if (!identical(a->left, b->left)) return false;
    return identical(a->right, b->right);
}
void runProblem4() {
    cout << "Tree A" << endl;
    Node* a = readTree();
    cout << "Tree B" << endl;
    Node* b = readTree();
    cout << "IDENTICAL(A, B) = " << (identical(a, b) ? "true" : "false") << endl;
}
```

Test Cases:

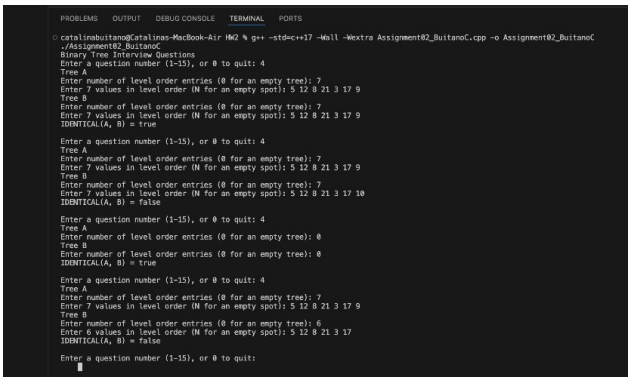


Figure 5: Test Cases 1 -4

Test 1 (given example)
Input: A and B from arr
Expected Output: true

Test 2 (given example)
Input: A from arr, C from {5, 12, 8, 21, 3, 17, 10}
Expected Output: false

Test 3 (edge case: both empty)
Input: NULL, NULL
Expected Output: true

Test 4 (edge case: different shape)
Input: A from arr, B from {5, 12, 8, 21, 3, 17} (no node 9)
Expected Output: false

Problem 5:

Description: Finding the smallest value is a basic search problem. It shows the difference between a plain binary tree, where the minimum could be anywhere, and a binary search tree, where its position is fixed.

Question: Return the smallest value in the tree. The tree is not a binary search tree, so every node must be checked. Also explain how the algorithm and its complexity would change for a BST.

Goal: Return the smallest value in the tree. Since it is not a binary search tree, the smallest value could be anywhere, so every node has to be checked. Expected output: 3.

Inputs: The root of a non-empty binary tree. Here it is the tree built from arr.

Outputs: One integer, the smallest value in the tree. Expected output: 3.

Constraints: Every node must be checked, because the tree has no ordering.

Assumptions: The tree has at least one node. Values can be negative. $+\infty$ is used as the result for an empty subtree so it never wins a comparison.

Edge Cases:

1. A single node, which should return its own value
2. Negative values, which must be handled by the comparison

3. The minimum at the root, at a leaf, or in either subtree
4. Duplicate values, which do not change the answer

Designing Solution: The smallest value in a tree is the smallest of three things: the root's value, the smallest value in the left subtree, and the smallest value in the right subtree. For the empty tree I return $+\infty$, because $+\infty$ never wins a min comparison, so a missing child does not affect the answer.

Pseudocode Problem 5: Minimum Value in a Binary Tree

Algorithm 5 MINVALUE

Input: node, the root of a tree or subtree

Output: the smallest value in that tree ($+\infty$ if empty)

Initial call: MINVALUE(root)

```
1: procedure MINVALUE(node)
2:     if node = NULL then
3:         return +∞
4:     end if
5:     leftMin ← MINVALUE(node.left)
6:     rightMin ← MINVALUE(node.right)
7:     return min(node.value, leftMin, rightMin)
8: end procedure
```

Explanation: The smallest value in a tree has to be either the root's value, the smallest value in the left subtree, or the smallest value in the right subtree, and line 7 takes the minimum of exactly those three. Returning $+\infty$ for NULL means a missing child can never be chosen as the minimum. Since the tree is not ordered, skipping any subtree could miss the answer, so visiting every node is necessary.

Worked Example:

Call #	Call	Lines executed	Returns
1	MINVALUE(5)	2, 5, 6, 7	min(5, 3, 8) = 3
2	MINVALUE(12)	2, 5, 6, 7	min(12, 21, 3) = 3
3	MINVALUE(21)	2, 5, 6, 7	min(21, $+\infty$, $+\infty$) = 21
4, 5	MINVALUE(N), MINVALUE(N) under 21	2, 3	$+\infty$, $+\infty$
6	MINVALUE(3)	2, 5, 6, 7	min(3, $+\infty$, $+\infty$) = 3
7, 8	MINVALUE(N), MINVALUE(N) under 3	2, 3	$+\infty$, $+\infty$
9	MINVALUE(8)	2, 5, 6, 7	min(8, 17, 9) = 8
10	MINVALUE(17)	2, 5, 6, 7	17
11, 12	MINVALUE(N), MINVALUE(N) under 17	2, 3	$+\infty$, $+\infty$
13	MINVALUE(9)	2, 5, 6, 7	9
14, 15	MINVALUE(N), MINVALUE(N) under 9	2, 3	$+\infty$, $+\infty$

Final output: 3. This matches the expected output.

Frequency Count:

Line	Operation	Cost	Frequency	Count for the example
2	test node = NULL	1	2n + 1	15
3	return $+\infty$	1	n + 1	8
5	call MINVALUE(node.left)	1	n	7
6	call MINVALUE(node.right)	1	n	7
7	return min of three	1	n	7
	Total T(n)		6n + 2	44

$T(n) = (2n + 1) + (n + 1) + n + n + n$ *(add the rows)*

$T(n) = 6n + 2$ *(collect terms)*

$T(7) = 6(7) + 2 = 44$ *(plug in n = 7; trace check: $7 \times 4 + 8 \times 2 = 28 + 16 = 44$)*

Time and Space Complexity: $T(n) = 6n + 2$, so the time is $\Theta(n)$ in the best and worst case. Because the tree is not ordered, the algorithm cannot skip any subtree, so even if the minimum is at the root it still has to check everything to be sure. The extra space is the recursion stack, $\Theta(h)$, which is $\Theta(\log n)$ for a balanced tree and $\Theta(n)$ for a chain, so a chain is the worst-case input.

Twist: In a BST every value in a left subtree is smaller than its parent, so the minimum is always the leftmost node. I would just keep going left until there is no left child, and never look at a right subtree:

Algorithm 5b BSTMIN

Input: root, the root of a non-empty BST

Output: the smallest value

Initial call: BSTMIN(root)

```
1: procedure BSTMIN(root)
2:     node ← root
3:     while node.left ≠ NULL do
4:         node ← node.left
5:     end while
6:     return node.value
7: end procedure
```

If d is the depth of the leftmost node, line 2 runs once, line 3 runs d + 1 times, line 4 runs d times and line 6 runs once:

$T = 1 + (d + 1) + d + 1 = 2d + 3$ *(add the rows)*

The worst case is d = h, so $T = 2h + 3$, which is $\Theta(h)$. That is $\Theta(\log n)$ for a balanced BST, but still $\Theta(n)$ if the BST is a left chain (for example, values inserted in decreasing order). The extra space becomes $\Theta(1)$, since it is a loop instead of recursion.

C++ Implementation:

```
// Problem 5
int minValue(Node* node) {
    if (node == nullptr) return INT_MAX;  // plays the role of +infinity
    int leftMin = minValue(node->left);
    int rightMin = minValue(node->right);
    return min({node->value, leftMin, rightMin});
}
```

```
// Twist: in a BST the minimum is the leftmost node
int bstMin(Node* root) {
    Node* node = root;
    while (node->left != nullptr) node = node->left;
    return node->value;
}
void runProblem5() {
    Node* root = readTree();
    while (root == nullptr) {
        cout << "The tree must have at least one node." << endl;
        root = readTree();
    }
    cout << "Minimum value: " << minValue(root) << endl;
    cout << "Leftmost node (the minimum only if the tree is a BST): " << bstMin(root) <<
    endl;
}
```

Test Cases:

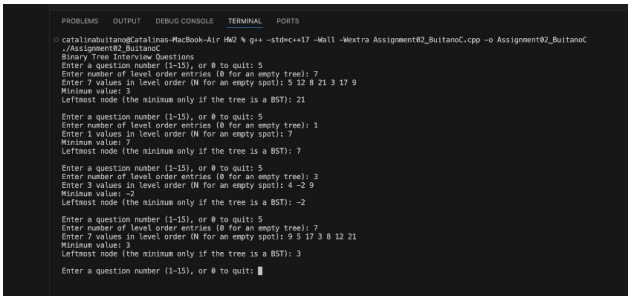


Figure 6: Test Cases 1-4

Test 1 (given example)
Input: the tree from arr
Expected Output: 3

Test 2 (edge case: single node)
Input: {7}
Expected Output: 7

Test 3 (edge case: negative value)
Input: {4, −2, 9}
Expected Output: −2

Test 4 (twist: BST version)
Input: the BST {9, 5, 17, 3, 8, 12, 21}
Expected Output: 3

Problem 6:

Description: Mirroring a tree is a common interview problem and is useful when a layout needs to be flipped, for example drawing a tree right to left, or checking whether a tree is symmetric.

Question: Swap the left and right child of every node, so the tree becomes its mirror image, and

return the root.

Goal: Swap the left and right child of every node so the tree becomes its mirror image, and return the root. Expected output: level order after inverting is 5, 8, 12, 9, 17, 3, 21.

Inputs: The root of a binary tree. Here it is the tree built from arr.

Outputs: The root of the same tree, now mirrored. Expected output: level order 5, 8, 12, 9, 17, 3, 21.

Constraints: The tree is changed in place, without building a new tree. Every node's children must be swapped, not just the root's.

Assumptions: The tree is not ordered. Inverting an empty tree returns NULL.

Edge Cases:

1. Empty tree, which should return NULL
2. A single node, which stays the same
3. A node with only one child, which moves the child to the other side
4. Applying INVERT twice, which should give back the original tree

Designing Solution: When I drew the mirror image, I noticed it is not enough to swap only the root's children, because the subtrees also have to be flipped inside. So at every node I swap the two child pointers using a temporary variable, then I call the same procedure on both children. It does not matter that I swap first and recurse second, since each subtree gets flipped no matter which side it ends up on.

Pseudocode Problem 6: Invert (Mirror) a Binary Tree

Algorithm 6 INVERT

Input: node, the root of a tree or subtree

Output: the root of the mirrored tree (the tree is changed in place)

Initial call: root ← INVERT(root)

```
 1: procedure INVERT(node)
 2:     if node = NULL then
 3:         return NULL
 4:     end if
 5:     temp ← node.left
 6:     node.left ← node.right
 7:     node.right ← temp
 8:     INVERT(node.left)
 9:     INVERT(node.right)
10:     return node
11: end procedure
```

Explanation: Lines 5 to 7 swap the two child pointers of the current node using temp, so no subtree is lost. Lines 8 and 9 then mirror both subtrees. A tree is mirrored exactly when its root's children are swapped and both subtrees are themselves mirrored, so doing this at every node produces the mirror image. The order of swapping and recursing does not matter, because each subtree is mirrored no matter which side it ends up on. NULL children are handled by the base

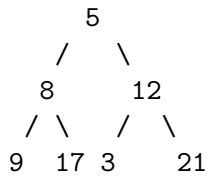
case on line 2.

Worked Example:

After the swap in lines 5 to 7, node.left is the old right child, so the recursion goes into the 8 side first.

Call #	Call	Lines executed	What happens	Returns
1	INVERT(5)	2, 5, 6, 7, 8, 9, 10	children become left = 8, right = 12	5
2	INVERT(8)	2, 5, 6, 7, 8, 9, 10	children become left = 9, right = 17	8
3	INVERT(9)	2, 5, 6, 7, 8, 9, 10	swaps two NULLs	9
4, 5	INVERT(N), INVERT(N) under 9	2, 3	—	NULL, NULL
6	INVERT(17)	2, 5, 6, 7, 8, 9, 10	swaps two NULLs	17
7, 8	INVERT(N), INVERT(N) under 17	2, 3	—	NULL, NULL
9	INVERT(12)	2, 5, 6, 7, 8, 9, 10	children become left = 3, right = 21	12
10	INVERT(3)	2, 5, 6, 7, 8, 9, 10	swaps two NULLs	3
11, 12	INVERT(N), INVERT(N) under 3	2, 3	—	NULL, NULL
13	INVERT(21)	2, 5, 6, 7, 8, 9, 10	swaps two NULLs	21
14, 15	INVERT(N), INVERT(N) under 21	2, 3	—	NULL, NULL

The tree after inverting:



Final output: level order 5, 8, 12, 9, 17, 3, 21. This matches the expected output.

Frequency Count:

Line	Operation	Cost	Frequency	Count for the example
2	test node = NULL	1	2n + 1	15
3	return NULL	1	n + 1	8
5	temp ← node.left	1	n	7
6	node.left ← node.right	1	n	7
7	node.right ← temp	1	n	7
8	call INVERT(node.left)	1	n	7
9	call INVERT(node.right)	1	n	7
10	return node	1	n	7
	Total T(n)		9n + 2	65

$T(n) = (2n + 1) + (n + 1) + 6n$ *(add the rows; lines 5 to 10 each run once per real node)*

$T(n) = 9n + 2$ *(collect terms)*

$T(7) = 9(7) + 2 = 65$ *(plug in n = 7; trace check: 7 calls × 7 lines + 8 calls × 2 lines = 49 + 16 = 65)*

Time and Space Complexity: $T(n) = 9n + 2$, so the time is $\Theta(n)$ in the best and worst case, since every node has to be swapped. The extra space is the recursion stack, $\Theta(h)$: $\Theta(\log n)$ for a

balanced tree in the best case and $\Theta(n)$ for a chain in the worst case. A chain is the worst-case input because it makes the stack n + 1 calls deep.

Twist: Inorder before: 21 12 3 5 17 8 9

Inorder after: 9 8 17 5 3 12 21

What I noticed is that the inorder after inverting is exactly the inorder before, read backwards. This happens because inorder visits left subtree, then node, then right subtree. After the swap, at every node the old right subtree is now on the left, so the traversal becomes old right subtree, then node, then old left subtree. That is the reverse of the original order at the root, and since every subtree was also mirrored, the same reversal happens all the way down, so the whole sequence comes out reversed.

C++ Implementation:

```
// Problem 6
Node* invertTree(Node* node) {
    if (node == nullptr) return nullptr;
    Node* temp = node->left;
    node->left = node->right;
    node->right = temp;
    invertTree(node->left);
    invertTree(node->right);
    return node;
}
void runProblem6() {
    Node* root = readTree();
    vector<int> before;
    inorder(root, before);
    root = invertTree(root);
    vector<int> after;
    inorder(root, after);
    cout << "Level order after inverting: "; printList(levelOrder(root)); cout << endl;
    cout << "Inorder before: "; printList(before); cout << endl;
    cout << "Inorder after:  "; printList(after); cout << endl;
}
```

Test Cases:

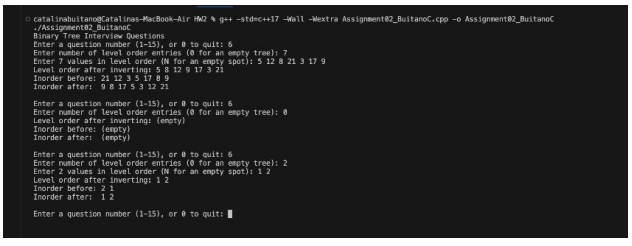


Figure 7: Test Cases 1-3

Test 1 (given example)
Input: the tree from arr
Expected Output: level order 5, 8, 12, 9, 17, 3, 21; inorder 9 8 17 5 3 12 21

Test 2 (edge case: empty tree)
Input: root = NULL
Expected Output: NULL

Test 3 (edge case: one child)
Input: {1, 2} (2 is the left child)
Expected Output: level order 1, 2 with 2 now the right child

Problem 7:

Description: Printing a tree level by level is how trees are usually displayed, and the right side view is a classic interview question about seeing which node is visible at each depth.

Question: Print the nodes of each level on a separate line. Then return the right side view: the last node of each level, which is what you see when looking at the tree from the right.

Goal: Print the nodes of each level on a separate line, then return the right side view, which is the last node of every level. Expected output: levels [5], [12, 8], [21, 3, 17, 9]; right side view 5, 8, 9.

Inputs: The root of a binary tree. Here it is the tree built from arr.

Outputs: One printed line per level, and a list with the last node of each level. Expected output: levels [5], [12, 8], [21, 3, 17, 9]; right side view 5, 8, 9.

Constraints: Nodes in a level are printed left to right. The right side view has exactly one value per level, h + 1 values in total.

Assumptions: The tree is not ordered. An empty tree prints nothing and returns an empty list.

Edge Cases:

1. Empty tree, which prints nothing and returns an empty list
2. A single node, which prints one line and returns that node
3. A tree where the rightmost node of a level is a left child, so the view is not just the right spine
4. A chain, which has n levels with one node each

Designing Solution: A normal level order traversal with a queue visits the nodes in the right order, but it does not know where one level stops and the next one starts. The trick I used is to read the size of the queue at the start of each level. At that moment the queue holds exactly the nodes of the current level, so if I dequeue exactly that many nodes, I finish the level and the queue is left holding only the next level. The last node I dequeue in each level (when i = size) is the one you would see from the right side.

Pseudocode Problem 7: Print Each Level on Its Own Line, and the Right Side View

Algorithm 7 LEVELSANDRIGHTVIEW

Input: root, the root of the tree

Output: prints each level on its own line and returns the right side view

Initial call: LEVELSANDRIGHTVIEW(root)

```
 1: procedure LEVELSANDRIGHTVIEW(root)
 2:     if root = NULL then
 3:         return empty list
 4:     end if
 5:     Q ← empty queue
 6:     ENQUEUE(Q, root)
 7:     rightView ← empty list
 8:     while Q is not empty do
 9:         size ← SIZE(Q)
10:         for i ← 1 to size do
11:             node ← DEQUEUE(Q)
12:             print node.value
13:             if i = size then
14:                 append node.value to rightView
15:             end if
16:             if node.left ≠ NULL then
17:                 ENQUEUE(Q, node.left)
18:             end if
19:             if node.right ≠ NULL then
20:                 ENQUEUE(Q, node.right)
21:             end if
22:         end for
23:         print new line
24:     end while
25:     return rightView
26: end procedure
```

Explanation: At the start of each pass of the while loop, the queue holds exactly the nodes of one level, because the previous pass took out every node of the level before and put in only their children. Reading SIZE(Q) on line 9 freezes that count, so the for loop removes exactly one level and nothing more. The nodes come out left to right because children are enqueued left before right. The node dequeued when i = size is the last one on that level, which is the rightmost node and the one seen from the right side.

Worked Example:

One row per dequeue, grouped by pass of the while loop (one pass = one level):

While pass	size	i	Dequeued	Printed	Right view	Enqueued	Queue after
start	—	—	—	—	[]	5	[5]
1 (level 0)	1	1	5	5, then new line	[5]	12, 8	[12, 8]
2 (level 1)	2	1	12	12	[5]	21, 3	[8, 21, 3]
		2	8	8, then new line	[5, 8]	17, 9	[21, 3, 17, 9]
3 (level 2)	4	1	21	21	[5, 8]	—	[3, 17, 9]
		2	3	3	[5, 8]	—	[17, 9]
		3	17	17	[5, 8]	—	[9]
		4	9	9, then new line	[5, 8, 9]	—	[]

The while test runs a 4th time, finds the queue empty, and the loop ends.

Final output: the lines 5 / 12 8 / 21 3 17 9, and the right side view 5, 8, 9. This matches the

expected output.

Frequency Count:

The tree has n nodes and h + 1 levels.

Line	Operation	Cost	Frequency	Count for the example
2	test root = NULL	1	1	1
5	Q ← empty queue	1	1	1
6	ENQUEUE(Q, root)	1	1	1
7	rightView ← empty list	1	1	1
8	test Q is not empty	1	h + 2	4
9	size ← SIZE(Q)	1	h + 1	3
10	for loop test	1	n + h + 1	10
11	DEQUEUE	1	n	7
12	print node.value	1	n	7
13	test i = size	1	n	7
14	append to rightView	1	h + 1	3
16	test node.left ≠ NULL	1	n	7
17	ENQUEUE left child	1	L	3
19	test node.right ≠ NULL	1	n	7
20	ENQUEUE right child	1	R	3
23	print new line	1	h + 1	3
25	return rightView	1	1	1
	Total T(n, h)		7n + 5h + 10	69

The for loop header runs size + 1 times on each level, so over all levels it runs (sum of sizes) + (number of levels) = n + (h + 1). Every node except the root is a child, so L + R = n − 1.

T = 5 + (h + 2) + (h + 1) + (n + h + 1) + 5n + (h + 1) + (n − 1) + (h + 1) *(the 5 is lines 2, 5, 6, 7, 25; the 5n is lines 11, 12, 13, 16, 19; lines 17 + 20 = n − 1)*

n terms: n + 5n + n = 7n *(group the n terms)*

h terms: h + h + h + h + h = 5h *(group the h terms)*

constants: 5 + 2 + 1 + 1 + 1 − 1 + 1 = 10 *(group the constants)*

T(n, h) = 7n + 5h + 10 *(put it together)*

T(7, 2) = 7(7) + 5(2) + 10 = 49 + 10 + 10 = 69 *(plug in n = 7, h = 2; matches the table column)*

Time and Space Complexity: The number of levels can never be more than the number of nodes, so h ≤ n − 1 and 5h is at most 5n. That makes the dominant part Θ(n), and the time is Θ(n) in the best and worst case. The worst case for the h term is a chain (h = n − 1), which gives T = 7n + 5(n − 1) + 10 = 12n + 5, still Θ(n). The extra space is the queue plus the right view list. The queue holds one level at a time, so a complete tree is the worst case at about n/2 nodes, Θ(n), and the right view list has h + 1 entries.

Twist: The 7n comes from the work done once per node: lines 11, 12, 13, 16, 19, the extra loop test for each node on line 10, and the n − 1 enqueues on lines 17 and 20. The 5h comes from the work done once per level: the while test (line 8), reading the size (line 9), the final for-loop test that ends each level (line 10), adding to the right view (line 14) and printing the new line (line 23). Each of those runs h + 1 times, which is where the h terms come from, and the leftover constants make up the + 10.

C++ Implementation:

```
// Problem 7
vector<int> levelsAndRightView(Node* root) {
    vector<int> rightView;
    if (root == nullptr) return rightView;
    queue<Node*> q;
    q.push(root);
    while (!q.empty()) {
        int size = (int)q.size();
        for (int i = 1; i <= size; i++) {
            Node* node = q.front(); q.pop();
            cout << node->value << " ";
            if (i == size) rightView.push_back(node->value);
            if (node->left != nullptr) q.push(node->left);
            if (node->right != nullptr) q.push(node->right);
        }
        cout << endl;
    }
    return rightView;
}
void runProblem7() {
    Node* root = readTree();
    cout << "Levels:" << endl;
    vector<int> view = levelsAndRightView(root);
    cout << "Right side view: "; printList(view); cout << endl;
}
```

Test Cases:

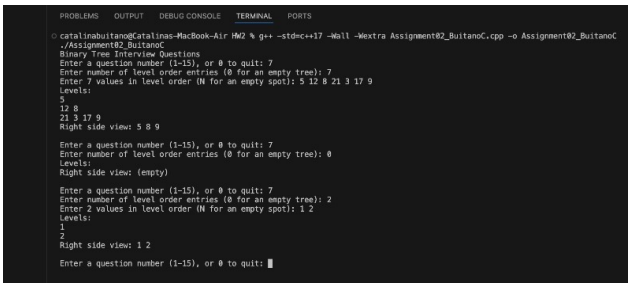


Figure 8: Test Cases 1-3

Test 1 (given example)
Input: the tree from arr
Expected Output: lines 5 / 12 8 / 21 3 17 9; right side view 5, 8, 9

Test 2 (edge case: empty tree)
Input: root = NULL
Expected Output: nothing printed; right side view empty

Test 3 (edge case: rightmost node is a left child)
Input: {1, 2}
Expected Output: lines 1 / 2; right side view 1, 2

Problem 8:

Description: Zigzag order is a variation of level order traversal that alternates direction on every level. It is a common interview question because it tests whether you can control the order of a traversal with the right data structure.

Question: Visit level 0 from left to right, level 1 from right to left, level 2 from left to right, and so on. Solve it with two stacks or with a queue plus reversing every second level, and compare the extra work.

Goal: Visit level 0 left to right, level 1 right to left, level 2 left to right, and keep alternating. Expected output: 5, 8, 12, 21, 3, 17, 9.

Inputs: The root of a binary tree. Here it is the tree built from arr.

Outputs: The values in zigzag order. Expected output: 5, 8, 12, 21, 3, 17, 9.

Constraints: Each level must be finished before the next one starts, and the direction has to alternate starting with left to right.

Assumptions: The tree is not ordered. An empty tree gives an empty output.

Edge Cases:

1. Empty tree, which gives an empty output
2. A single node, which outputs just the root
3. Levels with missing nodes, where the alternation still has to work
4. A deeper tree, where the direction has to flip back correctly on level 3

Designing Solution: I chose the two-stack method. A stack reverses the order of whatever is pushed into it, so if I pop the nodes of one level from a stack and push their children onto a second stack, the next level comes out in the opposite direction. The only thing I have to get right is the order I push the children. On a left-to-right level I push the left child first and then the right child, so the right child is on top and the next level pops right to left. On a right-to-left level I push right first and then left, so the next level pops left to right. When the current stack runs out, the level is done, so I swap the two stacks and flip the direction.

Pseudocode Problem 8: Zigzag (Spiral) Level Order Traversal

Algorithm 8 ZIGZAG

Input: root, the root of the tree

Output: prints the values in zigzag order

Initial call: ZIGZAG(root)

```
 1: procedure ZIGZAG(root)
 2:     if root = NULL then
 3:         return
 4:     end if
 5:     current ← empty stack
 6:     next ← empty stack
 7:     PUSH(current, root)
 8:     leftToRight ← true
 9:     while current is not empty do
10:         node ← POP(current)
11:         print node.value
12:         if leftToRight = true then
13:             if node.left ≠ NULL then
14:                 PUSH(next, node.left)
15:             end if
16:             if node.right ≠ NULL then
17:                 PUSH(next, node.right)
18:             end if
19:         else
20:             if node.right ≠ NULL then
21:                 PUSH(next, node.right)
22:             end if
23:             if node.left ≠ NULL then
24:                 PUSH(next, node.left)
25:             end if
26:         end if
27:         if current is empty then
28:             swap current and next
29:             leftToRight ← not leftToRight
30:         end if
31:     end while
32: end procedure
```

Explanation: The current stack holds one level and the next stack collects the next level. A stack gives back items in the reverse of the order they were pushed, so the order I push the children decides the direction of the next level. On a left-to-right level I push left then right, so the next level pops right to left; on a right-to-left level I push right then left, so the next level pops left to right. When current is empty the level is finished, so the stacks swap and the direction flips. Every node is pushed and popped exactly once, so every node is printed once.

Worked Example:

One row per pass of the while loop. Stacks are written bottom to top, so the last value is the top.

Pass	Direction	Popped	Printed	Pushed onto next	current after	next after	Swap?
start	L→R	—	—	—	[5]	[]	—
1	L→R	5	5	12 then 8	[]	[12, 8]	yes, now R→L
					[12, 8]	[]	
2	R→L	8	8	9 then 17	[12]	[9, 17]	no
3	R→L	12	12	3 then 21	[]	[9, 17, 3, 21]	yes, now L→R

Pass	Direction	Popped	Printed	Pushed onto next	current after	next after	Swap?
					[9, 17, 3, 21]	[]	
4	L→R	21	21	—	[9, 17, 3]	[]	no
5	L→R	3	3	—	[9, 17]	[]	no
6	L→R	17	17	—	[9]	[]	no
7	L→R	9	9	—	[]	[]	yes, now R→L

After pass 7 the while test finds current empty and the loop ends.

Final output: 5, 8, 12, 21, 3, 17, 9. This matches the expected output.

Frequency Count:

Let a be the number of nodes on left-to-right levels and b = n − a the number on right-to-left levels (a = 5, b = 2 here).

Line	Operation	Cost	Frequency	Count for the example
2	test root = NULL	1	1	1
5	current ← empty stack	1	1	1
6	next ← empty stack	1	1	1
7	PUSH(current, root)	1	1	1
8	leftToRight ← true	1	1	1
9	test current is not empty	1	n + 1	8
10	POP	1	n	7
11	print	1	n	7
12	test direction	1	n	7
13	test left (L→R levels)	1	a	5
14	PUSH left	1	left children of L→R nodes	1
16	test right (L→R levels)	1	a	5
17	PUSH right	1	right children of L→R nodes	1
20	test right (R→L levels)	1	b	2
21	PUSH right	1	right children of R→L nodes	2
23	test left (R→L levels)	1	b	2
24	PUSH left	1	left children of R→L nodes	2
27	test current is empty	1	n	7
28	swap stacks	1	h + 1	3
29	flip direction	1	h + 1	3
	Total T(n, h)		8n + 2h + 7	67

T = 5 + (n + 1) + 3n + (2a + 2b) + (pushes) + n + 2(h + 1) *(5 setup lines; lines 10, 11, 12 give 3n; lines 13, 16, 20, 23 give 2a + 2b)*

T = 5 + (n + 1) + 3n + 2n + (n − 1) + n + 2h + 2 *(a + b = n, and every node except the root is pushed once, so the pushes total n − 1)*

T(n, h) = 8n + 2h + 7 *(collect terms: 1 + 3 + 2 + 1 + 1 = 8 n's, and 5 + 1 − 1 + 2 = 7)*

T(7, 2) = 8(7) + 2(2) + 7 = 56 + 4 + 7 = 67 *(plug in n = 7, h = 2; matches the table column)*

Time and Space Complexity: Since h ≤ n − 1, the 2h term is never bigger than 2n, so the dominant term is 8n and the time is Θ(n) in the best and worst case. Every node is pushed once and popped once no matter what the tree looks like. The extra space is the two stacks, which together hold at most about two levels, so it is Θ(n) in the worst case (a complete tree, whose last level has about n/2 nodes) and Θ(1) for a chain.

Twist: The other way is to do the normal level order from Question 7, store each level in a list, and reverse the list on every odd level before printing it. Reversing a level of size s takes about s/2 swaps, and it means every node on an odd level gets touched one extra time after it was already dequeued. For our tree that is 1 extra swap (reversing [12, 8]), but on a big tree it adds up to $\Theta(n)$ extra work in the worst case. The two-stack version never moves a node twice: each node is pushed once and popped once, and the only extra work is the direction test on line 12 and the $2(h + 1)$ steps for swapping the stacks. So in my frequency count the two-stack method uses less extra work, even though both are $\Theta(n)$ overall.

C++ Implementation:

```
// Problem 8
vector<int> zigzag(Node* root) {
    vector<int> out;
    if (root == nullptr) return out;
    stack<Node*> current, next;
    current.push(root);
    bool leftToRight = true;
    while (!current.empty()) {
        Node* node = current.top(); current.pop();
        out.push_back(node->value);
        if (leftToRight) {
            if (node->left != nullptr) next.push(node->left);
            if (node->right != nullptr) next.push(node->right);
        } else {
            if (node->right != nullptr) next.push(node->right);
            if (node->left != nullptr) next.push(node->left);
        }
        if (current.empty()) {
            swap(current, next);
            leftToRight = !leftToRight;
        }
    }
    return out;
}
void runProblem8() {
    Node* root = readTree();
    cout << "Zigzag order: "; printList(zigzag(root)); cout << endl;
}
```

Test Cases:

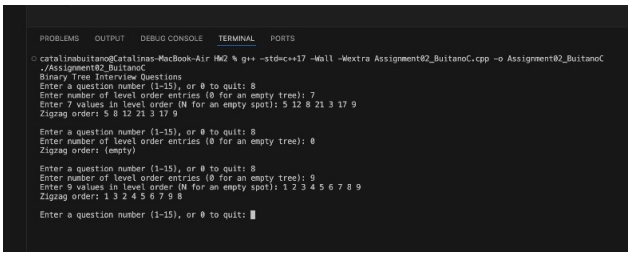


Figure 9: Test Cases 1-3

Test 1 (given example)

Input: the tree from arr
Expected Output: 5, 8, 12, 21, 3, 17, 9

Test 2 (edge case: empty tree)
Input: root = NULL
Expected Output: (empty)

Test 3 (edge case: four levels)
Input: {1, 2, 3, 4, 5, 6, 7, 8, 9}
Expected Output: 1, 3, 2, 4, 5, 6, 7, 9, 8

Problem 9:

Description: Complete trees are the shape used by binary heaps and by trees stored in arrays. Checking completeness confirms that a tree can be stored in an array with no gaps.

Question: A tree is complete if every level is full except possibly the last, and the last level is filled from the left. Return true or false.

Goal: Return true if every level is full except possibly the last, and the last level is filled from the left. Expected output: COMPLETE(P) = true and COMPLETE(Q) = false, where Q is {5, 12, 8, NULL, 3, 17, 9} (12 has no left child).

Inputs: The root of a binary tree. Tree P comes from arr; tree Q comes from {5, 12, 8, NULL, 3, 17, 9}, where 12 has no left child.

Outputs: A boolean. Expected output: COMPLETE(P) = true and COMPLETE(Q) = false.

Constraints: In level order, once a NULL child has been seen, no real node may appear after it.

Assumptions: An empty tree counts as complete. NULL children are put into the queue so that gaps can be noticed.

Edge Cases:

1. Empty tree, which is complete
2. A single node, which is complete
3. A last level filled only partly from the left, like {1, 2, 3, 4}, which is complete
4. A gap in the middle of a level, like tree Q, which is not complete

Designing Solution: I used the hint. In a complete tree, if you read the positions in level order, all the real nodes come first and all the empty spots come after them, with no gaps. So I do a level order traversal, but this time I also put the NULL children into the queue. The first time I take a NULL out of the queue I remember it with a flag. If I ever take out a real node after that, there was a gap, so the tree is not complete. If I get through the whole queue without that happening, it is complete.

Pseudocode Problem 9: Is the Tree Complete?

Algorithm 9 COMPLETE

Input: root, the root of the tree

Output: true if the tree is complete, otherwise false

Initial call: COMPLETE(root)

```
 1: procedure COMPLETE(root)
 2:     if root = NULL then
 3:         return true
 4:     end if
 5:     Q ← empty queue
 6:     ENQUEUE(Q, root)
 7:     seenNull ← false
 8:     while Q is not empty do
 9:         node ← DEQUEUE(Q)
10:         if node = NULL then
11:             seenNull ← true
12:         else
13:             if seenNull = true then
14:                 return false
15:             end if
16:             ENQUEUE(Q, node.left)
17:             ENQUEUE(Q, node.right)
18:         end if
19:     end while
20:     return true
21: end procedure
```

Explanation: Reading the positions of a tree in level order, a tree is complete exactly when all the real nodes come before all the empty spots. The algorithm reads the positions in level order, including NULLs, and remembers the first NULL with seenNull. If a real node comes out after that, there is a gap, so line 14 correctly returns false. If the queue empties without that happening, every NULL came after every real node, so line 20 correctly returns true.

Worked Example:

COMPLETE(P), one row per pass (N means a NULL entry in the queue):

Pass	Dequeued	seenNull	Enqueued	Queue after the pass
start	—	false	5	[5]
1	5	false	12, 8	[12, 8]
2	12	false	21, 3	[8, 21, 3]
3	8	false	17, 9	[21, 3, 17, 9]
4	21	false	N, N	[3, 17, 9, N, N]
5	3	false	N, N	[17, 9, N, N, N, N]
6	17	false	N, N	[9, N, N, N, N, N, N]
7	9	false	N, N	[N, N, N, N, N, N, N, N]
8	N	true	—	7 N's left
9 to 15	N (one each pass)	true	—	6, 5, 4, 3, 2, 1, 0 N's left

The queue empties without a real node ever coming after a NULL, so line 20 returns true.

COMPLETE(Q):

Pass	Dequeued	seenNull	Enqueued	Queue after the pass
start	—	false	5	[5]
1	5	false	12, 8	[12, 8]

Pass	Dequeued	seenNull	Enqueued	Queue after the pass
2	12	false	N, 3	[8, N, 3]
3	8	false	17, 9	[N, 3, 17, 9]
4	N	true	—	[3, 17, 9]
5	3	true	— (returns false on line 14)	stops

Final output: COMPLETE(P) = true and COMPLETE(Q) = false. Both match the expected output.

Frequency Count:

The formula is for a complete tree of n nodes, since that is when the loop runs all the way. Every real node and every NULL pointer goes through the queue once, which is n + (n + 1) = 2n + 1 dequeues.

Line	Operation	Cost	Frequency	Count (P)	Count (Q)
2	test root = NULL	1	1	1	1
5	Q ← empty queue	1	1	1	1
6	ENQUEUE(Q, root)	1	1	1	1
7	seenNull ← false	1	1	1	1
8	test Q is not empty	1	2n + 2	16	5
9	DEQUEUE	1	2n + 1	15	5
10	test node = NULL	1	2n + 1	15	5
11	seenNull ← true	1	n + 1	8	1
13	test seenNull	1	n	7	4
14	return false	1	0	0	1
16	ENQUEUE left	1	n	7	3
17	ENQUEUE right	1	n	7	3
20	return true	1	1	1	0
	Total T(n)		10n + 10	80	31

$T(n) = 5 + (2n + 2) + (2n + 1) + (2n + 1) + (n + 1) + n + n + n$ *(the 5 is lines 2, 5, 6, 7, 20)*

$T(n) = 5 + 10n + 5$ *(add the n terms: 2 + 2 + 2 + 1 + 1 + 1 + 1 = 10; add the constants: 2 + 1 + 1 + 1 = 5)*

$T(n) = 10n + 10$ *(collect terms)*

$T(7) = 10(7) + 10 = 80$ *(plug in n = 7; matches the P column)*

For Q (n = 6): 4 setup lines + 5 + 5 + 5 + 1 + 4 + 1 + 3 + 3 = 31. It is much smaller because it stops on pass 5.

Time and Space Complexity: The dominant term is 10n, so the worst case is $\Theta(n)$. The worst-case input is a tree that really is complete, because the algorithm cannot say true until it has emptied the whole queue, including all n + 1 NULLs. The best case is $\Theta(1)$, when a gap shows up right at the top: for example, if the root has a right child but no left child, the function returns false on the third pass. The extra space is the queue, which holds up to about one level plus its NULL children, which is $\Theta(n)$ for a complete tree (it reached 8 entries for P).

Twist: INSERT from the lecture scans in level order and puts the new node in the first open spot it finds. That spot is always the leftmost empty position on the last level (or the first spot of a new level if the last one is full), which is exactly position n in the array, so the filled positions stay 0 to n with no gap. DELETE replaces the value being removed with the value of the deepest,

rightmost node, which is the last node in level order (position n − 1), and then removes that last node. Removing the last position also leaves positions 0 to n − 2 filled with no gap. Since both operations only ever add or remove at the end of the level order, there is never a NULL in front of a real node, so the tree stays complete.

C++ Implementation:

```
// Problem 9
bool isComplete(Node* root) {
    if (root == nullptr) return true;
    queue<Node*> q;
    q.push(root);
    bool seenNull = false;
    while (!q.empty()) {
        Node* node = q.front(); q.pop();
        if (node == nullptr) {
            seenNull = true;
        } else {
            if (seenNull) return false;
            q.push(node->left);
            q.push(node->right);
        }
    }
    return true;
}
void runProblem9() {
    Node* root = readTree();
    cout << "COMPLETE = " << (isComplete(root) ? "true" : "false") << endl;
}
```

Test Cases:

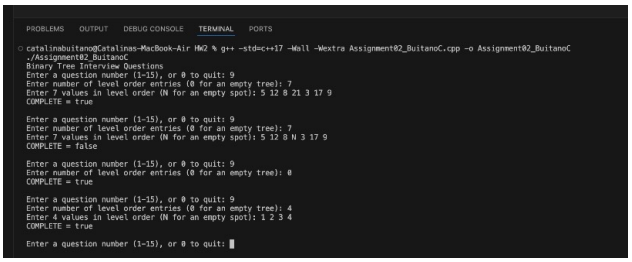


Figure 10: Test Case 1-3

Test 1 (given example)
Input: tree P from arr
Expected Output: true

Test 2 (given example)
Input: tree Q from {5, 12, 8, NULL, 3, 17, 9}
Expected Output: false

Test 3 (edge case: empty tree)
Input: root = NULL
Expected Output: true

Test 4 (edge case: partial last level)
Input: {1, 2, 3, 4}
Expected Output: true

Problem 10:

Description: The lowest common ancestor is used in file systems (the deepest shared folder), family trees and network routing. It is one of the most common binary tree interview questions.

Question: Given two values a and b in the tree, return their lowest common ancestor: the deepest node that has both a and b in its subtree (a node counts as being in its own subtree). Solve it with FINDPATH or with a single recursive pass and compare the frequency counts.

Goal: Given two values a and b that are in the tree, return the deepest node that has both of them in its subtree (a node counts as being in its own subtree). Expected output: LCA(21, 3) = 12, LCA(21, 9) = 5, LCA(17, 8) = 8.

Inputs: The root of a binary tree and two values a and b. Here it is the tree built from arr.

Outputs: The node that is the lowest common ancestor. Expected output: LCA(21, 3) = 12, LCA(21, 9) = 5, LCA(17, 8) = 8.

Constraints: A node counts as an ancestor of itself, so if a is above b, the answer is a.

Assumptions: Both a and b are in the tree, and all values are distinct. The tree is not a BST, so values cannot be used to choose a side.

Edge Cases:

1. One value is an ancestor of the other, like LCA(17, 8) = 8
2. The two values are in different subtrees of the root, like LCA(21, 9) = 5
3. a = b, which returns that node
4. Both values are leaves, which makes the search visit the most nodes

Designing Solution: I went with the single recursive pass. Each call asks its subtree: did you find a or b? If the current node is a or b, it returns itself right away, because whatever is below it, this node is already the highest point of interest on that side. Otherwise it asks both children. If both sides found something, then a is on one side and b is on the other, so the current node is where their paths split, which makes it the LCA. If only one side found something, I pass that result up. This works because the problem says both values are in the tree.

Pseudocode Problem 10: Lowest Common Ancestor

Algorithm 10 LCA

Input: node, the root of a tree or subtree; a and b, the two values (both in the tree)

Output: the lowest common ancestor of a and b, or NULL if neither is in this subtree

Initial call: LCA(root, a, b)

```
 1: procedure LCA(node, a, b)
 2:     if node = NULL then
 3:         return NULL
 4:     end if
 5:     if node.value = a or node.value = b then
 6:         return node
 7:     end if
 8:     leftResult ← LCA(node.left, a, b)
 9:     rightResult ← LCA(node.right, a, b)
10:     if leftResult ≠ NULL and rightResult ≠ NULL then
11:         return node
12:     end if
13:     if leftResult ≠ NULL then
14:         return leftResult
15:     end if
16:     return rightResult
17: end procedure
```

Explanation: Each call returns the a or b node it found in its subtree, or the LCA if both were found below it, or NULL if neither is there. If the current node is a or b, it is the highest point of interest on that path, so returning it is correct even if the other value is below it (that is the ancestor case). If both sides return something, a and b are on different sides, so the current node is the deepest node with both below it. If only one side returns something, both values are on that side, so the answer from that side is passed up unchanged.

Worked Example:

LCA(21, 3), every call:

Call #	Call	Lines executed	Returns
1	LCA(5)	2, 5, 8, 9, 10, 13, 14	left = 12, right = NULL, so 12
2	LCA(12)	2, 5, 8, 9, 10, 11	left = 21, right = 3, both found, so 12
3	LCA(21)	2, 5, 6	21 (it is a)
4	LCA(3)	2, 5, 6	3 (it is b)
5	LCA(8)	2, 5, 8, 9, 10, 13, 16	NULL, NULL, so NULL
6	LCA(17)	2, 5, 8, 9, 10, 13, 16	NULL
7, 8	LCA(N), LCA(N) under 17	2, 3	NULL, NULL
9	LCA(9)	2, 5, 8, 9, 10, 13, 16	NULL
10, 11	LCA(N), LCA(N) under 9	2, 3	NULL, NULL

Result: 12. Notice that LCA(21) and LCA(3) stop at line 6, so their NULL children are never called.

The other two queries, shortened:

- LCA(21, 9): the left side of 5 returns 21 (found inside 12's subtree) and the right side returns 9 (found inside 8's subtree). Both are non-NULL at node 5, so line 11 returns 5.
- LCA(17, 8): the left side of 5 searches 12, 21, 3 and returns NULL. The right side reaches 8, which equals b, so it returns 8 immediately without looking at 17. At node 5, only the right result is non-NULL, so line 16 returns 8. This is correct because 17 is inside 8's subtree.

Final output: LCA(21, 3) = 12, LCA(21, 9) = 5, LCA(17, 8) = 8. All three match the expected output.

Frequency Count:

LCA can stop early, so I wrote the frequencies using: v = real nodes visited, m = nodes that matched a or b (stop at line 6), and s = 1 if some node returns on line 11, otherwise 0. The v − m nodes that do not match each make 2 calls, so the total number of calls is 1 + 2(v − m). For LCA(21, 3): v = 7, m = 2, s = 1.

Line	Operation	Cost	Frequency	Count for LCA(21, 3)
2	test node = NULL	1	1 + 2(v − m)	11
3	return NULL	1	1 + 2(v − m) − v	4
5	test node is a or b	1	v	7
6	return node	1	m	2
8	call LCA(node.left)	1	v − m	5
9	call LCA(node.right)	1	v − m	5
10	test both found	1	v − m	5
11	return node	1	s	1
13	test left found	1	v − m − s	4
14	return leftResult	1	ℓ (left result passed up)	1
16	return rightResult	1	r (right result passed up), with ℓ + r = v − m − s	3
	Total T		9(v − m) + m − s + 2	48

T = [1 + 2(v − m)] + [1 + 2(v − m) − v] + v + m + 3(v − m) + s + 2(v − m − s) *(add the rows, counting lines 14 and 16 together)*

T = 2 + (2 + 2 + 3 + 2)(v − m) − v + v + m + s − 2s *(group the (v − m) terms)*

T = 9(v − m) + m − s + 2 *(simplify: −v + v cancels and s − 2s = −s)*

T = 9(7 − 2) + 2 − 1 + 2 = 45 + 3 = 48 *(plug in v = 7, m = 2, s = 1; trace check: four 7-line calls (5, 8, 17, 9) + one 6-line call (12) + two 3-line calls (21, 3) + four 2-line NULL calls = 28 + 6 + 6 + 8 = 48)*

Worst case: v can be at most n and m − s is at most 2, so T ≤ 9n + 2.

Time and Space Complexity: T ≤ 9n + 2, so the worst case is Θ(n). The worst case happens when the two values are leaves, or are deep and far apart, so the search has to visit (almost) every node before both are found. The best case is Θ(1), when the root itself is a or b: line 6 returns right away after 3 steps. The extra space is the recursion stack, Θ(h).

Twist: With FINDPATH, I would call it twice to get the root-to-a path and the root-to-b path, then walk both paths from the root and return the last node they share. For LCA(21, 3) the paths are [5, 12, 21] and [5, 12, 3], and the last shared node is 12. Each FINDPATH is a search that can visit all n nodes in the worst case, so that is about two full traversals, plus Θ(h) more steps to compare the paths, and Θ(h) extra space to store them. The single pass finds both values in one traversal, so its count is about half of the path method (at most 9n + 2), and it only needs the recursion stack. Both methods are Θ(n), but the single pass does less work.

C++ Implementation:

```
// Problem 10
Node* lca(Node* node, int a, int b) {
    if (node == nullptr) return nullptr;
```

```
    if (node->value == a || node->value == b) return node;
    Node* leftResult = lca(node->left, a, b);
    Node* rightResult = lca(node->right, a, b);
    if (leftResult != nullptr && rightResult != nullptr) return node;
    if (leftResult != nullptr) return leftResult;
    return rightResult;
}
void runProblem10() {
    Node* root = readTree();
    int a, b;
    cout << "Enter the two values a and b: ";
    cin >> a >> b;
    Node* answer = lca(root, a, b);
    if (answer == nullptr) cout << "Neither value is in the tree." << endl;
    else cout << "LCA(" << a << ", " << b << ") = " << answer->value << endl;
}
```

Test Cases:

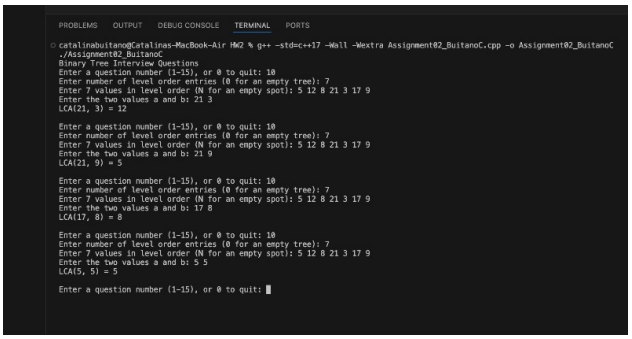


Figure 11: Test Cases 1-4

Test 1 (given example)
Input: LCA(21, 3) on the tree from arr
Expected Output: 12

Test 2 (given example)
Input: LCA(21, 9)
Expected Output: 5

Test 3 (given example: ancestor case)
Input: LCA(17, 8)
Expected Output: 8

Test 4 (edge case: a = b)
Input: LCA(5, 5)
Expected Output: 5

Problem 11:

Description: Rebuilding a tree from its traversals is how a tree can be sent or stored as two flat lists and recovered later. It also shows why one traversal alone is not enough to describe a tree.

Question: Given the preorder and inorder traversals of a tree with distinct values, rebuild the tree

and return its root. Also find the worst-case frequency count when the inorder list is searched each time, and show how a lookup table brings it down to $\Theta(n)$.

Goal: Given the preorder and inorder lists of a tree with distinct values, rebuild the tree and return its root. Input: preorder = 5, 12, 21, 3, 8, 17, 9 and inorder = 21, 12, 3, 5, 17, 8, 9. Expected output: the tree from arr (level order 5, 12, 8, 21, 3, 17, 9; postorder 21, 3, 12, 17, 9, 8, 5).

Inputs: Two lists of the same n distinct values: preorder = 5, 12, 21, 3, 8, 17, 9 and inorder = 21, 12, 3, 5, 17, 8, 9.

Outputs: The root of the rebuilt tree. Expected output: the tree from arr, with level order 5, 12, 8, 21, 3, 17, 9 and postorder 21, 3, 12, 17, 9, 8, 5.

Constraints: All values must be distinct, otherwise the root's position in the inorder list would not be unique.

Assumptions: Both lists come from the same tree and have the same length. Empty lists give an empty tree.

Edge Cases:

1. Empty lists, which return NULL
2. A single value, which returns one node
3. A chain (for example every node only has a left child), which is the worst case for searching the inorder list
4. The root at either end of the inorder list, which makes one subtree empty

Designing Solution: Preorder always lists the root first, and inorder lists the left subtree, then the root, then the right subtree. So the first preorder value is the root, and when I find that value in the inorder list, everything to its left is the left subtree and everything to its right is the right subtree. I keep a pointer preIdx into the preorder list and move it forward by one each time I make a node, because preorder builds the root, then the whole left subtree, then the whole right subtree, which is the same order my recursion runs in. To avoid searching the inorder list every time, I first build a lookup table index[value] = position of that value in inorder, so finding the split point is one step.

Pseudocode Problem 11: Build a Tree From Its Preorder and Inorder Traversals

Algorithm 11a BUILDTREE

Input: pre and in, the preorder and inorder lists of n distinct values

Output: the root of the rebuilt tree

Initial call: BUILDTREE(pre, in, n)

```
1: procedure BUILDTREE(pre, in, n)
2:     for i ← 0 to n − 1 do
3:         index[in[i]] ← i
4:     end for
5:     preIdx ← 0
6:     return BUILD(0, n − 1)
7: end procedure
```

Algorithm 11b BUILD

Input: lo and hi, the range of the inorder list that this subtree covers (pre, index and preIdx are shared)

Output: the root of the subtree for that range (NULL if the range is empty)

Initial call: BUILD(0, n − 1), made by BUILDTREE

```
 1: procedure BUILD(lo, hi)
 2:     if lo > hi then
 3:         return NULL
 4:     end if
 5:     rootValue ← pre[preIdx]
 6:     preIdx ← preIdx + 1
 7:     node ← new node with value rootValue
 8:     m ← index[rootValue]
 9:     node.left ← BUILD(lo, m − 1)
10:     node.right ← BUILD(m + 1, hi)
11:     return node
12: end procedure
```

Explanation: The first value in a preorder range is always the root of that subtree, and in the inorder list everything left of the root belongs to the left subtree and everything right of it belongs to the right subtree. Line 8 finds that split point, and lines 9 and 10 build the two subtrees from the two halves of the range. preIdx moves forward once per node, and since the recursion builds the root, then the whole left subtree, then the whole right subtree, it uses the preorder list in exactly the order it was written. An empty range (lo > hi) means there is no subtree, so it returns NULL.

Worked Example:

First the loop in BUILDTREE fills the lookup table from the inorder list:

Value	21	12	3	5	17	8	9
index	0	1	2	3	4	5	6

Then the BUILD calls (an empty range runs lines 2, 3 and returns NULL; a real call runs lines 2, 5, 6, 7, 8, 9, 10, 11):

Call #	Call	preIdx used	rootValue	m	Left range, right range	Returns
1	BUILD(0, 6)	0	5	3	(0, 2), (4, 6)	node 5
2	BUILD(0, 2)	1	12	1	(0, 0), (2, 2)	node 12
3	BUILD(0, 0)	2	21	0	(0, −1), (1, 0)	node 21
4, 5	BUILD(0, −1), BUILD(1, 0)	—	—	—	empty	NULL, NULL
6	BUILD(2, 2)	3	3	2	(2, 1), (3, 2)	node 3
7, 8	BUILD(2, 1), BUILD(3, 2)	—	—	—	empty	NULL, NULL
9	BUILD(4, 6)	4	8	5	(4, 4), (6, 6)	node 8
10	BUILD(4, 4)	5	17	4	(4, 3), (5, 4)	node 17
11, 12	BUILD(4, 3), BUILD(5, 4)	—	—	—	empty	NULL, NULL
13	BUILD(6, 6)	6	9	6	(6, 5), (7, 6)	node 9
14, 15	BUILD(6, 5), BUILD(7, 6)	—	—	—	empty	NULL, NULL

So 5 has children 12 and 8, 12 has children 21 and 3, and 8 has children 17 and 9.

Final output: this is the tree from arr. Its level order is 5, 12, 8, 21, 3, 17, 9 and its postorder is 21, 3, 12, 17, 9, 8, 5, which both match the check in the expected output.

Frequency Count:

BUILD is called once for each of the n nodes and once for each of the n + 1 empty spots, so 2n + 1 calls.

Line	Operation	Cost	Frequency	Count for the example
BUILDTREE 2	for loop test	1	n + 1	8
BUILDTREE 3	index[in[i]] ← i	1	n	7
BUILDTREE 5	preIdx ← 0	1	1	1
BUILDTREE 6	call BUILD(0, n − 1)	1	1	1
BUILD 2	test lo > hi	1	2n + 1	15
BUILD 3	return NULL	1	n + 1	8
BUILD 5	rootValue ← pre[preIdx]	1	n	7
BUILD 6	preIdx ← preIdx + 1	1	n	7
BUILD 7	make new node	1	n	7
BUILD 8	m ← index[rootValue]	1	n	7
BUILD 9	call BUILD(lo, m − 1)	1	n	7
BUILD 10	call BUILD(m + 1, hi)	1	n	7
BUILD 11	return node	1	n	7
	Total T(n)		12n + 5	89

BUILDTREE = (n + 1) + n + 1 + 1 = 2n + 3 *(add the four BUILDTREE rows)*

BUILD = (2n + 1) + (n + 1) + 7n = 10n + 2 *(lines 5 to 11 are 7 lines that each run n times)*

T(n) = (2n + 3) + (10n + 2) = 12n + 5 *(add both procedures)*

T(7) = 12(7) + 5 = 89 *(plug in n = 7; check: BUILDTREE is 8 + 7 + 1 + 1 = 17, BUILD is 7 calls × 8 lines + 8 calls × 2 lines = 56 + 16 = 72, and 17 + 72 = 89)*

Time and Space Complexity: T(n) = 12n + 5, so the time is $\Theta(n)$ in the best and worst case: the table is built in one pass and every node is made once with a one-step lookup. The extra space is the lookup table, $\Theta(n)$, plus the recursion stack, $\Theta(h)$, which is $\Theta(n)$ for a chain. The worst-case input for space is a chain-shaped tree, since the stack gets as deep as the tree.

Twist: If I replace line 8 with a linear search through in[lo..hi], each search can cost up to the size of the range. The worst case is a chain where the root is always at the far end of its range, for example a tree where every node only has a left child: preorder is a1, a2, . . . , an and inorder is an, . . . , a2, a1. Then the root of the first call is the last of n positions, the next one is the last of n − 1 positions, and so on:

search steps = n + (n − 1) + . . . + 2 + 1 *(one full scan of each shrinking range)*

search steps = n(n + 1) / 2 *(sum of the first n integers)*

search steps = $n^2/2 + n/2$ *(expand)*

So line 8 alone becomes $\Theta(n^2)$, and the whole algorithm is $\Theta(n^2)$ in the worst case. With the lookup table, building it costs n + 1 + n = 2n + 1 steps one time, and then line 8 costs only 1 step per node, n in total, so everything stays at 12n + 5, which is $\Theta(n)$. For n = 7 the search version would need up to 7(8)/2 = 28 search steps on a chain, compared with 7 lookups.

C++ Implementation:

```
// Problem 11
Node* buildRange(const vector<int>& pre, unordered_map<int, int>& index,
                 int& preIdx, int lo, int hi) {
    if (lo > hi) return nullptr;
    int rootValue = pre[preIdx];
    preIdx++;
    Node* node = new Node(rootValue);
    int m = index[rootValue];
    node->left = buildRange(pre, index, preIdx, lo, m - 1);
    node->right = buildRange(pre, index, preIdx, m + 1, hi);
    return node;
}

Node* buildTree(const vector<int>& pre, const vector<int>& in) {
    int n = (int)in.size();
    unordered_map<int, int> index;
    for (int i = 0; i < n; i++) index[in[i]] = i;
    int preIdx = 0;
    return buildRange(pre, index, preIdx, 0, n - 1);
}
void runProblem11() {
    int n;
    cout << "Enter number of values: ";
    cin >> n;
    while (n < 0) {
        cout << "Number of values cannot be negative. Enter number of values: ";
        cin >> n;
    }
    vector<int> pre(n), in(n);
    cout << "Enter the preorder list: ";
    for (int i = 0; i < n; i++) cin >> pre[i];
    cout << "Enter the inorder list: ";
    for (int i = 0; i < n; i++) cin >> in[i];
    Node* root = buildTree(pre, in);
    vector<int> post;
    postorder(root, post);
    cout << "Level order: "; printList(levelOrder(root)); cout << endl;
    cout << "Postorder:   "; printList(post); cout << endl;
}
```

Test Cases:

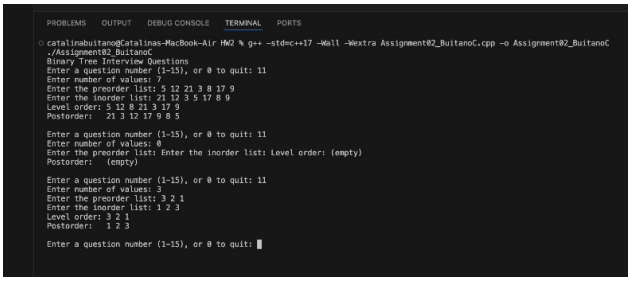


Figure 12: Test Cases 1-3

Test 1 (given example)
Input: preorder 5 12 21 3 8 17 9, inorder 21 12 3 5 17 8 9
Expected Output: the tree from arr (level order 5 12 8 21 3 17 9, postorder 21 3 12 17 9 8 5)

Test 2 (edge case: empty lists)
Input: preorder and inorder empty
Expected Output: NULL

Test 3 (edge case: left chain)
Input: preorder 3 2 1, inorder 1 2 3
Expected Output: 3 with left child 2 with left child 1

Problem 12:

Description: The diameter is the longest distance between any two nodes. It measures how spread out a tree is, which matters for things like the worst-case delay between two points in a network shaped like a tree.

Question: The diameter is the number of edges on the longest path between any two nodes, and the path does not have to pass through the root. Compute the height and the diameter in the same pass and prove with the frequency count that it is $\Theta(n)$.

Goal: Return the number of edges on the longest path between any two nodes. The path does not have to go through the root. Expected output: 4 (for example 21, 12, 5, 8, 17).

Inputs: The root of a binary tree. Here it is the tree built from arr.

Outputs: One integer, the number of edges on the longest path. Expected output: 4 (for example the path 21, 12, 5, 8, 17).

Constraints: The path can bend at any node, not only the root. It is measured in edges.

Assumptions: The diameter of a single node is 0. For an empty tree the function returns 0. Height uses height(NULL) $= -1$, the same as Question 1.

Edge Cases:

1. A single node, which has diameter 0
2. A chain of n nodes, which has diameter $n - 1$ and is the case that makes the simple solution $\Theta(n^2)$
3. A longest path that does not go through the root
4. An empty tree, which returns 0

Designing Solution: Every path in a tree has one highest node, where it goes down on the left side and down on the right side. So the longest path that bends at a given node is (height of left subtree + 1) + (height of right subtree + 1), where the +1's are the two edges from the node down to its children. The simple solution calls HEIGHT separately at every node, which repeats a lot of work. Instead, I compute the height bottom-up, and while I already have lh and rh for a node, I also check whether the path bending there is the longest so far and save it in a variable best. That way each node is visited once and I get the height and the diameter in the same pass. I use height(NULL) $= -1$, so for a leaf lh + rh + 2 = 0, which is correct.

Pseudocode Problem 12: Diameter of a Binary Tree

Algorithm 12a DIAMETER

Input: root, the root of the tree

Output: the number of edges on the longest path between two nodes

Initial call: DIAMETER(root)

```
1: procedure DIAMETER(root)
2:     best ← 0
3:     HEIGHT(root)
4:     return best
5: end procedure
```

Algorithm 12b HEIGHT

Input: node, the root of a tree or subtree (best is shared with DIAMETER)

Output: the height of that subtree (−1 if empty); also updates best

Initial call: HEIGHT(root), made by DIAMETER

```
1: procedure HEIGHT(node)
2:     if node = NULL then
3:         return −1
4:     end if
5:     lh ← HEIGHT(node.left)
6:     rh ← HEIGHT(node.right)
7:     best ← max(best, lh + rh + 2)
8:     return 1 + max(lh, rh)
9: end procedure
```

Explanation: Every path in a tree has one highest node where it bends, so the longest path is the longest path that bends at some node. The longest path bending at a node goes down its left side as far as possible and down its right side as far as possible, which is (lh + 1) + (rh + 1) = lh + rh + 2 edges. Line 7 checks that value at every node and keeps the biggest one in best, so after the traversal best is the diameter. Line 8 returns the height so the parent can use it, which is why no subtree is visited twice.

Worked Example:

The calls are listed in the order they start, but best is updated on line 7 when each call finishes, so I also show the order they finish.

Call #	Call	Lines executed	lh, rh	best after line 7	Returns	Finishes
1	HEIGHT(5)	2, 5, 6, 7, 8	1, 1	max(2, 1 + 1 + 2) = 4	2	7th
2	HEIGHT(12)	2, 5, 6, 7, 8	0, 0	max(0, 0 + 0 + 2) = 2	1	3rd
3	HEIGHT(21)	2, 5, 6, 7, 8	−1, −1	max(0, 0) = 0	0	1st
4, 5	HEIGHT(N), HEIGHT(N) under 21	2, 3	—	—	−1, −1	—
6	HEIGHT(3)	2, 5, 6, 7, 8	−1, −1	max(0, 0) = 0	0	2nd
7, 8	HEIGHT(N), HEIGHT(N) under 3	2, 3	—	—	−1, −1	—
9	HEIGHT(8)	2, 5, 6, 7, 8	0, 0	max(2, 2) = 2	1	6th
10	HEIGHT(17)	2, 5, 6, 7, 8	−1, −1	max(2, 0) = 2	0	4th
11, 12	HEIGHT(N), HEIGHT(N) under 17	2, 3	—	—	−1, −1	—
13	HEIGHT(9)	2, 5, 6, 7, 8	−1, −1	max(2, 0) = 2	0	5th
14, 15	HEIGHT(N), HEIGHT(N) under 9	2, 3	—	—	−1, −1	—

DIAMETER runs lines 2, 3, 4 and returns best = 4. The longest path bends at the root: 1 + 1 edges down the left (5 to 12 to 21) and 1 + 1 down the right (5 to 8 to 17).

Final output: 4. This matches the expected output.

Frequency Count:

Line	Operation	Cost	Frequency	Count for the example
DIAMETER 2	best ← 0	1	1	1
DIAMETER 3	call HEIGHT(root)	1	1	1
DIAMETER 4	return best	1	1	1
HEIGHT 2	test node = NULL	1	2n + 1	15
HEIGHT 3	return −1	1	n + 1	8
HEIGHT 5	call HEIGHT(node.left)	1	n	7
HEIGHT 6	call HEIGHT(node.right)	1	n	7
HEIGHT 7	update best	1	n	7
HEIGHT 8	return 1 + max(lh, rh)	1	n	7
	Total T(n)		7n + 5	54

T(n) = 3 + (2n + 1) + (n + 1) + 4n *(3 lines in DIAMETER; lines 5 to 8 of HEIGHT run once per real node)*

T(n) = 7n + 5 *(collect terms: 2 + 1 + 4 = 7 n's, and 3 + 1 + 1 = 5)*

T(7) = 7(7) + 5 = 54 *(plug in n = 7; check: 3 + 7 calls × 5 lines + 8 calls × 2 lines = 3 + 35 + 16 = 54)*

Time and Space Complexity: T(n) = 7n + 5, so the time is Θ(n) in the best and worst case, because every node is visited exactly once whatever the shape. The extra space is the recursion stack, Θ(h): Θ(log n) for a balanced tree and Θ(n) for a chain, so a chain is the worst-case input for space.

Twist: The simple solution computes diameter(node) = max(HEIGHT(left) + HEIGHT(right) + 2, diameter(left), diameter(right)), and each HEIGHT call walks the whole subtree again. On a chain of n nodes, the HEIGHT calls at the top node cover n − 1 nodes, at the next node n − 2, and so on:

work ≈ (n − 1) + (n − 2) + . . . + 1 + 0 *(one HEIGHT walk under every node of the chain)*

work ≈ n(n − 1) / 2 *(sum of the first n − 1 integers)*

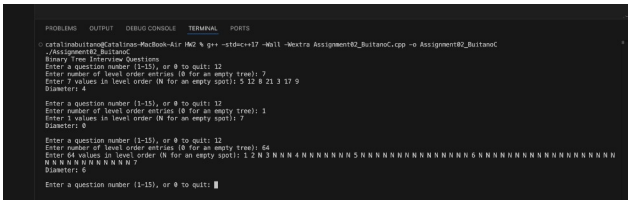
That is $\Theta(n^2)$. In my version, line 7 reuses the lh and rh values that the recursion already returned, so no subtree is ever walked twice. My frequency count shows every line runs at most 2n + 1 times, which adds up to T(n) = 7n + 5 on any tree, including a chain: for n = 7 that is 54 steps, while the simple version would repeat about 7(6)/2 = 21 extra node visits on a chain just for the heights.

C++ Implementation:

```
// Problem 12
int heightAndDiameter(Node* node, int& best) {
    if (node == nullptr) return -1;
    int lh = heightAndDiameter(node->left, best);
    int rh = heightAndDiameter(node->right, best);
    best = max(best, lh + rh + 2);
    return 1 + max(lh, rh);
}
```

```
int diameter(Node* root) {
    int best = 0;
    heightAndDiameter(root, best);
    return best;
}
void runProblem12() {
    Node* root = readTree();
    cout << "Diameter: " << diameter(root) << endl;
}
```

Test Cases:


PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
catalinabuitrago@Catalinas-MacBook-Air HW2 % g++ -std=c++17 -Wall -Wextra Assignment02_BuitanoC.cpp -o Assignment02_BuitanoC
./Assignment02_BuitanoC
Binary Tree Interview Questions
Enter a question number (1-15), or 0 to quit: 12
Enter number of level order entries (0 for an empty tree): 7
Enter 7 values in level order (N for an empty spot): 5 12 8 21 3 17 9
Diameter: 4
Enter a question number (1-15), or 0 to quit: 12
Enter number of level order entries (0 for an empty tree): 1
Enter 1 values in level order (N for an empty spot): 7
Diameter: 0
Enter a question number (1-15), or 0 to quit: 12
Enter number of level order entries (0 for an empty tree): 64
Enter 64 values in level order (N for an empty spot): 1 2 N 3 N N N 4 N N N N N N N 5 N N N N N N N N N N N N N N N N 6 N N N N N N N N N N N N N N N N

Assumptions: Values are integers and N is never a real value. DESERIALIZE only receives lists made by SERIALIZE.

Edge Cases:

1. Empty tree, which serializes to just N and deserializes back to NULL
2. A single node, which serializes to three items (value N N)
3. A chain, where most of the N markers are on one side
4. Negative values, which still read back correctly

Designing Solution: A plain preorder list is not enough to rebuild a tree, because it does not say where a subtree ends. Writing N for every NULL child fixes that, since every node's left and right sides are now spelled out completely. SERIALIZE is just a preorder traversal that also writes the NULLs. DESERIALIZE reads the list in the same preorder order with a shared position pos: read one item, and if it is N return NULL; otherwise make a node with that value, then build its left subtree from the next items, then its right subtree. Because both procedures go in exactly the same order, each item lands in the right place.

Pseudocode Problem 13: Serialize and Deserialize a Binary Tree

Algorithm 13a SERIALIZE

Input: node, the root of a tree or subtree; list, the output list

Output: the values in preorder with N for each NULL child, appended to list

Initial call: list ← empty list; SERIALIZE(root, list)

```
1: procedure SERIALIZE(node, list)
2:     if node = NULL then
3:         append N to list
4:         return
5:     end if
6:     append node.value to list
7:     SERIALIZE(node.left, list)
8:     SERIALIZE(node.right, list)
9: end procedure
```

Algorithm 13b DESERIALIZE

Input: list, made by SERIALIZE (pos is a shared position in the list)

Output: the root of the rebuilt tree

Initial call: pos ← 0; root ← DESERIALIZE(list)

```
 1: procedure DESERIALIZE(list)
 2:     item ← list[pos]
 3:     pos ← pos + 1
 4:     if item = N then
 5:         return NULL
 6:     end if
 7:     node ← new node with value item
 8:     node.left ← DESERIALIZE(list)
 9:     node.right ← DESERIALIZE(list)
10:     return node
11: end procedure
```

Explanation: SERIALIZE writes each node before its subtrees and writes N for each missing child, so the list fully describes where every subtree starts and ends. DESERIALIZE reads the list in the same preorder: it makes a node, then rebuilds the left subtree from the next items, then the right subtree. Each N tells it a subtree is empty, so it knows exactly when to stop and move to the right side. Because both procedures go in the same order and pos moves forward one item per call, every item is used exactly once and the rebuilt tree has the same shape and values.

Worked Example:

SERIALIZE (a real node runs lines 2, 6, 7, 8; a NULL runs lines 2, 3, 4):

Call #	Call	Appends	List so far
1	SERIALIZE(5)	5	5
2	SERIALIZE(12)	12	5 12
3	SERIALIZE(21)	21	5 12 21
4, 5	NULL, NULL under 21	N, N	5 12 21 N N
6	SERIALIZE(3)	3	5 12 21 N N 3
7, 8	NULL, NULL under 3	N, N	5 12 21 N N 3 N N
9	SERIALIZE(8)	8	... 3 N N 8
10	SERIALIZE(17)	17	... 8 17
11, 12	NULL, NULL under 17	N, N	... 8 17 N N
13	SERIALIZE(9)	9	... 17 N N 9
14, 15	NULL, NULL under 9	N, N	... 9 N N

Result: 5 12 21 N N 3 N N 8 17 N N 9 N N (15 items).

DESERIALIZE on that list (a value runs lines 2, 3, 4, 7, 8, 9, 10; an N runs lines 2, 3, 4, 5):

Call #	pos read	Item	Action	Returns
1	0	5	make 5, build left, then right	node 5 (left 12, right 8)
2	1	12	make 12, build left, then right	node 12 (left 21, right 3)
3	2	21	make 21, build left, then right	node 21 (leaf)
4	3	N	—	NULL (left of 21)
5	4	N	—	NULL (right of 21)
6	5	3	make 3, build left, then right	node 3 (leaf)
7	6	N	—	NULL (left of 3)
8	7	N	—	NULL (right of 3)
9	8	8	make 8, build left, then right	node 8 (left 17, right 9)
10	9	17	make 17, build left, then right	node 17 (leaf)
11	10	N	—	NULL (left of 17)

Call #	pos read	Item	Action	Returns
12	11	N	—	NULL (right of 17)
13	12	9	make 9, build left, then right	node 9 (leaf)
14	13	N	—	NULL (left of 9)
15	14	N	—	NULL (right of 9)

Final output: SERIALIZE gives 5 12 21 N N 3 N N 8 17 N N 9 N N, and DESERIALIZE rebuilds 5 with children 12 and 8, 12 with children 21 and 3, and 8 with children 17 and 9, which is the tree from arr. Both match the expected output.

Frequency Count:

SERIALIZE:

Line	Operation	Cost	Frequency	Count for the example
2	test node = NULL	1	2n + 1	15
3	append N	1	n + 1	8
4	return	1	n + 1	8
6	append node.value	1	n	7
7	call SERIALIZE(node.left)	1	n	7
8	call SERIALIZE(node.right)	1	n	7
	Total T(n)		7n + 3	52

$T(n) = (2n + 1) + (n + 1) + (n + 1) + n + n + n = 7n + 3$ *(add the rows and collect terms)*

$T(7) = 7(7) + 3 = 52$ *(plug in n = 7; check: 7 calls × 4 lines + 8 calls × 3 lines = 28 + 24 = 52)*

DESERIALIZE:

Line	Operation	Cost	Frequency	Count for the example
2	item ← list[pos]	1	2n + 1	15
3	pos ← pos + 1	1	2n + 1	15
4	test item = N	1	2n + 1	15
5	return NULL	1	n + 1	8
7	make new node	1	n	7
8	call for left child	1	n	7
9	call for right child	1	n	7
10	return node	1	n	7
	Total T(n)		11n + 4	81

$T(n) = 3(2n + 1) + (n + 1) + 4n$ *(lines 2, 3, 4 run once per list item; lines 7 to 10 once per value)*

$T(n) = 6n + 3 + n + 1 + 4n = 11n + 4$ *(expand and collect terms)*

$T(7) = 11(7) + 4 = 81$ *(plug in n = 7; check: 7 calls × 7 lines + 8 calls × 4 lines = 49 + 32 = 81)*

Time and Space Complexity: SERIALIZE is $7n + 3$ and DESERIALIZE is $11n + 4$, so both are $\Theta(n)$ in the best and worst case: each one handles every one of the $2n + 1$ list items exactly once, whatever the shape of the tree. The extra space is the list, $\Theta(n)$, plus the recursion stack, $\Theta(h)$, which becomes $\Theta(n)$ for a chain, so a chain is the worst-case input for space.

Twist: The list has one item for every real node and one N for every NULL child pointer. A tree with n nodes has 2n child pointers (two per node). Every node except the root is pointed to by exactly one of them, so $n - 1$ pointers are used and the rest are NULL:

NULL pointers = $2n - (n - 1) = n + 1$ *(total pointers minus the ones that point to a node)*

list length = $n + (n + 1) = 2n + 1$ *(values plus N markers)*

For our tree that is $2(7) + 1 = 15$ items, which matches the list. I used this fact directly in both frequency counts: each item comes from exactly one call, so both procedures make exactly $2n + 1$ calls, which is where the $2n + 1$ rows come from, and the $n + 1$ N items are where the $n + 1$ rows come from.

C++ Implementation:

```
// Problem 13
void serialize(Node* node, vector<string>& list) {
    if (node == nullptr) {
        list.push_back("N");
        return;
    }
    list.push_back(to_string(node->value));
    serialize(node->left, list);
    serialize(node->right, list);
}

Node* deserialize(const vector<string>& list, int& pos) {
    string item = list[pos];
    pos++;
    if (item == "N") return nullptr;
    Node* node = new Node(stoi(item));
    node->left = deserialize(list, pos);
    node->right = deserialize(list, pos);
    return node;
}
void runProblem13() {
    Node* root = readTree();
    vector<string> list;
    serialize(root, list);
    cout << "SERIALIZE: ";
    for (const string& item : list) cout << item << " ";
    cout << "(" << list.size() << " items)" << endl;
    int pos = 0;
    Node* rebuilt = deserialize(list, pos);
    cout << "DESERIALIZE level order: "; printList(levelOrder(rebuilt)); cout << endl;
}
```

Test Cases:

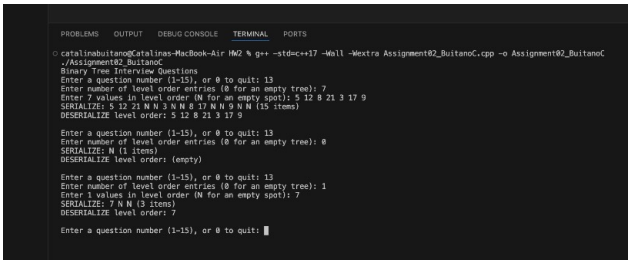


Figure 14: Test Cases 1-3

Test 1 (given example)
Input: the tree from arr
Expected Output: 5 12 21 N N 3 N N 8 17 N N 9 N N, and DESERIALIZE gives back the same tree

Test 2 (edge case: empty tree)
Input: root = NULL
Expected Output: N, and DESERIALIZE gives back NULL

Test 3 (edge case: single node)
Input: {7}
Expected Output: 7 N N (3 = 2(1) + 1 items)

Problem 14:

Description: Binary search trees only work if their ordering rule holds everywhere, so checking validity is important after building or changing one. It is a classic interview question because the obvious check is wrong.

Question: Return true if every node is larger than all values in its left subtree and smaller than all values in its right subtree. Pass down an allowed range (low, high). Then give a second solution using an inorder traversal and compare both frequency counts and best cases.

Goal: Return true if every node is larger than every value in its left subtree and smaller than every value in its right subtree. Expected output: ISBST(P) = false for the tree from arr, and ISBST(S) = true for S = {9, 5, 17, 3, 8, 12, 21} in level order.

Inputs: The root of a binary tree. Tree P comes from arr; tree S comes from {9, 5, 17, 3, 8, 12, 21}.

Outputs: A boolean. Expected output: ISBST(P) = false and ISBST(S) = true (its inorder is 3, 5, 8, 9, 12, 17, 21).

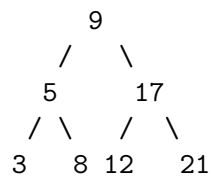
Constraints: Every node must be strictly inside its range, so duplicate values make the tree invalid.

Assumptions: An empty tree is a valid BST. $-\infty$ and $+\infty$ are used as the starting range.

Edge Cases:

1. Empty tree, which is valid
2. A value that is fine compared to its parent but breaks the rule with a higher ancestor, like 6 in the right subtree of 10
3. Duplicate values, which are not allowed
4. A violation right below the root, which is the best case

Tree S:



Designing Solution: At first I thought checking each node against its two children would be enough, but the hint is right that it is not. For example, a value deep in the left subtree could still

be bigger than the root, and comparing only parent and child would miss it. So I pass down an allowed range (low, high). The root can be anything, so it starts with $(-\infty, +\infty)$. When I go left, every value must be smaller than the current node, so the new high is node.value. When I go right, every value must be bigger, so the new low is node.value. If any node falls outside its range, the tree is not a BST. I stop as soon as I find a problem.

Pseudocode Problem 14: Is the Tree a Valid Binary Search Tree?

Algorithm 14 ISBST

Input: node, the root of a tree or subtree; low and high, the allowed range (both exclusive)

Output: true if every value in this subtree is inside the range and the subtree is a BST, otherwise false

Initial call: ISBST(root, $-\infty$, $+\infty$)

```
 1: procedure ISBST(node, low, high)
 2:     if node = NULL then
 3:         return true
 4:     end if
 5:     if node.value ≤ low or node.value ≥ high then
 6:         return false
 7:     end if
 8:     if ISBST(node.left, low, node.value) = false then
 9:         return false
10:     end if
11:     return ISBST(node.right, node.value, high)
12: end procedure
```

Explanation: When I go left from a node, every value below must be smaller than that node, so the node's value becomes the new high. When I go right, every value below must be bigger, so the node's value becomes the new low. The range a node gets therefore contains every constraint from all of its ancestors, not just its parent, which is what the parent-child check misses. If every node is inside its range, every node is bigger than everything in its left subtree and smaller than everything in its right subtree, which is the BST definition.

Worked Example:

ISBST(S), every call (a real node runs lines 2, 5, 8, 11; a NULL runs lines 2, 3):

Call #	Call (node, low, high)	In range?	Returns
1	(9, $-\infty$, $+\infty$)	yes	true
2	(5, $-\infty$, 9)	yes	true
3	(3, $-\infty$, 5)	yes	true
4, 5	NULL, NULL under 3	—	true, true
6	(8, 5, 9)	yes	true
7, 8	NULL, NULL under 8	—	true, true
9	(17, 9, $+\infty$)	yes	true
10	(12, 9, 17)	yes	true
11, 12	NULL, NULL under 12	—	true, true
13	(21, 17, $+\infty$)	yes	true
14, 15	NULL, NULL under 21	—	true, true

ISBST(P), the tree from arr:

Call #	Call (node, low, high)	Lines executed	Returns
1	(5, $-\infty$, $+\infty$)	2, 5, 8, 9	false (left side failed)
2	(12, $-\infty$, 5)	2, 5, 6	false, because $12 \geq 5$

Final output: ISBST(P) = false and ISBST(S) = true. Both match the expected output.

Frequency Count:

The formula is for the worst case, a valid BST, where every call runs to the end. The last column shows P, which fails right away.

Line	Operation	Cost	Frequency (valid BST)	Count (S)	Count (P)
2	test node = NULL	1	2n + 1	15	2
3	return true	1	n + 1	8	0
5	test value out of range	1	n	7	2
6	return false	1	0	0	1
8	call ISBST(left) and test	1	n	7	1
9	return false	1	0	0	1
11	call ISBST(right) and return	1	n	7	0
	Total T(n)		6n + 2	44	7

T(n) = (2n + 1) + (n + 1) + n + n + n *(add the rows; lines 6 and 9 never run on a valid BST)*

T(n) = 6n + 2 *(collect terms)*

T(7) = 6(7) + 2 = 44 *(plug in n = 7; check: 7 calls × 4 lines + 8 calls × 2 lines = 28 + 16 = 44)*

Time and Space Complexity: The dominant term is 6n, so the worst case is Θ(n). The worst-case input is a tree that really is a valid BST, because the algorithm can only return true after checking every node and every NULL. The best case is $\Theta(1)$: if the root's left child is already out of range, like in P, it stops after 7 steps. The extra space is the recursion stack, Θ(h), which is Θ(n) for a chain.

Twist: The inorder traversal of a BST lists its values in increasing order (S gives 3, 5, 8, 9, 12, 17, 21). So I can do an inorder traversal while remembering the previous value, and return false the first time a value is not bigger than the one before it.

Algorithm 14b ISBSTINORDER

Input: node, the root of a tree or subtree (prev is shared and starts at $-\infty$)

Output: true if the inorder values are strictly increasing, otherwise false

Initial call: prev $\leftarrow -\infty$; ISBSTINORDER(root)

```
 1: procedure ISBSTINORDER(node)
 2:     if node = NULL then
 3:         return true
 4:     end if
 5:     if ISBSTINORDER(node.left) = false then
 6:         return false
 7:     end if
 8:     if node.value ≤ prev then
 9:         return false
10:     end if
11:     prev ← node.value
12:     return ISBSTINORDER(node.right)
13: end procedure
```

Line	Frequency (valid BST)	Count (S)	Count (P)
2	2n + 1	15	5
3	n + 1	8	2
5	n	7	3
6	0	0	1
8	n	7	2
9	0	0	1
11	n	7	1
12	n	7	1
Total	7n + 2	51	16

$T(n) = (2n + 1) + (n + 1) + 4n = 7n + 2$ *(lines 5, 8, 11, 12 each run once per node on a valid BST)*

$T(7) = 7(7) + 2 = 51$ *(plug in n = 7)*

Comparison: on a valid BST the range method is $6n + 2 = 44$ steps and the inorder method is $7n + 2 = 51$, so both are $\Theta(n)$ and the range method is slightly cheaper because it has one less line per node. The best cases are different. The range method can stop at the root's child, which is $\Theta(1)$ (7 steps for P). The inorder method cannot check anything until it has walked all the way down to the leftmost node, so its best case is $\Theta(d)$, where d is the depth of the leftmost node: for P it had to go down 5, 12, 21 before finding the problem at 12 (16 steps), and on a balanced tree that walk is $\Theta(\log n)$.

C++ Implementation:

```
// Problem 14
// long long bounds stand in for -infinity and +infinity
bool isBST(Node* node, long long low, long long high) {
    if (node == nullptr) return true;
    if (node->value <= low || node->value >= high) return false;
    if (!isBST(node->left, low, node->value)) return false;
    return isBST(node->right, node->value, high);
}

// Twist: inorder version, prev starts at -infinity
bool isBSTInorder(Node* node, long long& prev) {
```

```
    if (node == nullptr) return true;
    if (!isBSTInorder(node->left, prev)) return false;
    if (node->value <= prev) return false;
    prev = node->value;
    return isBSTInorder(node->right, prev);
}
void runProblem14() {
    Node* root = readTree();
    long long prev = LLONG_MIN;
    cout << "ISBST (range method): " << (isBST(root, LLONG_MIN, LLONG_MAX) ? "true" : "
    false") << endl;
    cout << "ISBST (inorder method): " << (isBSTInorder(root, prev) ? "true" : "false")
    << endl;
}
```

Test Cases:

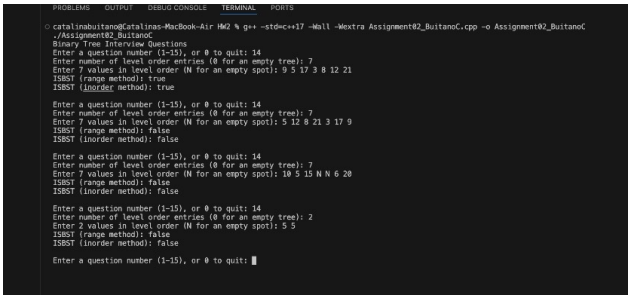


Figure 15: Test Cases 1-4

Test 1 (given example)
Input: tree S from {9, 5, 17, 3, 8, 12, 21}
Expected Output: true

Test 2 (given example)
Input: tree P from arr
Expected Output: false

Test 3 (edge case: ancestor rule)
Input: {10, 5, 15, NULL, NULL, 6, 20}
Expected Output: false (6 is in the right subtree of 10)

Test 4 (edge case: duplicate)
Input: {5, 5}
Expected Output: false

Problem 15:

Description: Vertical order shows a tree column by column, like looking at it from above. It is used when drawing trees and in interview problems like the top view and bottom view of a tree.

Question: Give the root horizontal distance 0, a left child its parent's distance − 1 and a right child its parent's distance + 1. Print the nodes column by column from the leftmost distance to the rightmost, and inside a column print top to bottom and left to right (use a level order traversal).

Goal: Give the root horizontal distance 0, a left child its parent's distance − 1, and a right child its parent's distance + 1. Print the nodes column by column from the leftmost distance to the rightmost, and inside a column print top to bottom and left to right. Expected output: −2: 21 | −1: 12 | 0: 5, 3, 17 | +1: 8 | +2: 9.

Inputs: The root of a binary tree. Here it is the tree built from arr.

Outputs: One printed line per column. Expected output: −2: 21 | −1: 12 | 0: 5, 3, 17 | +1: 8 | +2: 9.

Constraints: Columns are printed from the smallest distance to the largest. Inside a column the order is top to bottom, and left to right for nodes on the same level.

Assumptions: The queue holds (node, distance) pairs. A hash map (or an array indexed by d + n − 1) groups the values by distance in one step each.

Edge Cases:

1. Empty tree, which prints nothing
2. A single node, which prints one column (distance 0)
3. Several nodes in the same column on the same level, like 3 and 17
4. A chain going one way, which gives n columns

Designing Solution: Labeling the tree by hand, I got $5 \to 0$, $12 \to -1$, $8 \to +1$, $21 \to -2$, $3 \to 0$, $17 \to 0$ and $9 \to +2$. Inside column 0 the order has to be 5 first (it is higher up), then 3 before 17 (same level, 3 is further left). A level order traversal already visits nodes top to bottom and left to right, so if I append each node to its column's list in the order I dequeue it, every column ends up in the right order automatically. The queue holds (node, distance) pairs so each child can work out its own distance from its parent's. To group the nodes I use a map from distance to a list of values, and I keep track of the smallest and largest distance I have seen so I know which columns to print and in what order.

Pseudocode Problem 15: Vertical Order Traversal

Algorithm 15 VERTICALORDER

Input: root, the root of the tree

Output: prints each column from the leftmost distance to the rightmost

Initial call: VERTICALORDER(root)

```
 1: procedure VERTICALORDER(root)
 2:     if root = NULL then
 3:         return
 4:     end if
 5:     cols ← empty map from distance to list of values
 6:     Q ← empty queue of (node, distance) pairs
 7:     ENQUEUE(Q, (root, 0))
 8:     minD ← 0
 9:     maxD ← 0
10:     while Q is not empty do
11:         (node, d) ← DEQUEUE(Q)
12:         append node.value to cols[d]
13:         minD ← min(minD, d)
14:         maxD ← max(maxD, d)
15:         if node.left ≠ NULL then
16:             ENQUEUE(Q, (node.left, d − 1))
17:         end if
18:         if node.right ≠ NULL then
19:             ENQUEUE(Q, (node.right, d + 1))
20:         end if
21:     end while
22:     for d ← minD to maxD do
23:         print "distance", d
24:         for each value v in cols[d] do
25:             print v
26:         end for
27:     end for
28: end procedure
```

Explanation: Every node gets the right distance because it is computed from its parent's distance, starting from 0 at the root. Level order visits nodes top to bottom, and left to right inside a level, so appending each value to its column as it is dequeued puts every column in the order the question asks for. minD and maxD record the smallest and largest distance seen, and since distances change by 1 from parent to child, every column between them has at least one node, so the printing loop never prints an empty column.

Worked Example:

One row per pass of the while loop (queue front to back after the pass):

Pass	Dequeued (node, d)	cols after the pass	minD, maxD	Queue after the pass
start	—	{}	0, 0	[(5, 0)]
1	(5, 0)	0: [5]	0, 0	[(12, −1), (8, +1)]
2	(12, −1)	−1: [12], 0: [5]	−1, 0	[(8, +1), (21, −2), (3, 0)]
3	(8, +1)	−1: [12], 0: [5], +1: [8]	−1, +1	[(21, −2), (3, 0), (17, 0), (9, +2)]
4	(21, −2)	−2: [21] added	−2, +1	[(3, 0), (17, 0), (9, +2)]
5	(3, 0)	0: [5, 3]	−2, +1	[(17, 0), (9, +2)]
6	(17, 0)	0: [5, 3, 17]	−2, +1	[(9, +2)]
7	(9, +2)	+2: [9] added	−2, +2	[]

Then the printing loop goes from d = −2 to d = +2:

d	Printed
−2	21
−1	12
0	5, 3, 17
+1	8
+2	9

Final output: −2: 21 | −1: 12 | 0: 5, 3, 17 | +1: 8 | +2: 9. This matches the expected output.

Frequency Count:

Let c be the number of columns, c = maxD − minD + 1 (c = 5 here).

Line	Operation	Cost	Frequency	Count for the example
2	test root = NULL	1	1	1
5	cols ← empty map	1	1	1
6	Q ← empty queue	1	1	1
7	ENQUEUE(root, 0)	1	1	1
8	minD ← 0	1	1	1
9	maxD ← 0	1	1	1
10	test Q is not empty	1	n + 1	8
11	DEQUEUE	1	n	7
12	append to cols[d] (grouping)	1	n	7
13	update minD	1	n	7
14	update maxD	1	n	7
15	test node.left ≠ NULL	1	n	7
16	ENQUEUE left child	1	L	3
18	test node.right ≠ NULL	1	n	7
19	ENQUEUE right child	1	R	3
22	outer for loop test	1	c + 1	6
23	print distance	1	c	5
24	inner for loop test	1	n + c	12
25	print value	1	n	7
	Total T(n, c)		10n + 3c + 7	92

The inner loop on line 24 runs (size of column) + 1 times for each column, which adds up to n + c over all columns. As before, L + R = n − 1.

T = 6 + (n + 1) + 6n + (n − 1) + (c + 1) + c + (n + c) + n *(6 setup lines; lines 11, 12, 13, 14, 15, 18 give 6n; lines 16 + 19 give n − 1)*

n terms: 1 + 6 + 1 + 1 + 1 = 10, so 10n *(group the n terms)*

c terms: 1 + 1 + 1 = 3, so 3c *(group the c terms)*

constants: 6 + 1 − 1 + 1 = 7 *(group the constants)*

T(n, c) = 10n + 3c + 7 *(put it together)*

T(7, 5) = 10(7) + 3(5) + 7 = 70 + 15 + 7 = 92 *(plug in n = 7, c = 5; matches the table column)*

Time and Space Complexity: Every column holds at least one node, so c ≤ n, and the 3c term is at most 3n. That gives T ≤ 13n + 7, so the time is Θ(n) in the best and worst case, assuming

the map is a hash map or an array indexed by d + (n − 1), so that line 12 is one step. (If I used an ordered map like std::map instead, each insert would cost $\Theta(\log c)$ and the total would be $\Theta(n \log n)$.) The worst case for the c term is a chain that always goes the same direction, which makes c = n. The extra space is $\Theta(n)$ for the queue and the columns.

Twist: The grouping cost is line 12, which runs once per node, n times, and the printing cost is lines 22 to 25, which come to (c + 1) + c + (n + c) + n = 2n + 3c + 1. Both are included in the total above. A tree of n nodes can have at most n columns, because each column needs at least one node in it, and it reaches exactly n when the tree is a chain where every node is a left child (distances $0, -1, -2, \ldots, -(n-1)$) or every node is a right child. The fewest columns for $n \geq 2$ is 2, for example a root with a single child. Our tree has c = 5 columns for n = 7 nodes.

C++ Implementation:

```
// Problem 15
void verticalOrder(Node* root) {
    if (root == nullptr) return;
    unordered_map<int, vector<int>> cols;
    queue<pair<Node*, int>> q;
    q.push({root, 0});
    int minD = 0, maxD = 0;
    while (!q.empty()) {
        Node* node = q.front().first;
        int d = q.front().second;
        q.pop();
        cols[d].push_back(node->value);
        minD = min(minD, d);
        maxD = max(maxD, d);
        if (node->left != nullptr) q.push({node->left, d - 1});
        if (node->right != nullptr) q.push({node->right, d + 1});
    }
    for (int d = minD; d <= maxD; d++) {
        cout << "distance " << d << ":";
        for (int v : cols[d]) cout << " " << v;
        cout << endl;
    }
}
void runProblem15() {
    Node* root = readTree();
    cout << "Vertical order:" << endl;
    verticalOrder(root);
}
```

Test Cases:

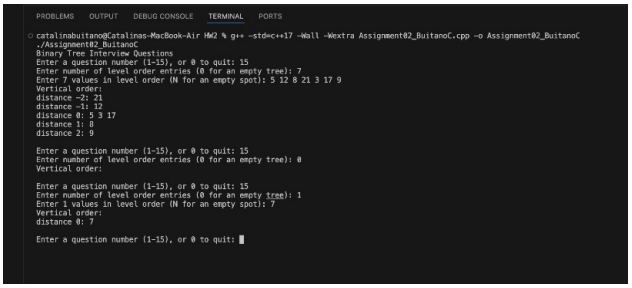


Figure 16: Test Cases 1-3

Test 1 (given example)
Input: the tree from arr
Expected Output: distance -2: 21 / -1: 12 / 0: 5 3 17 / $+1$: 8 / $+2$: 9

Test 2 (edge case: empty tree)
Input: root = NULL
Expected Output: (nothing printed)

Test 3 (edge case: single node)
Input: {7}
Expected Output: distance 0: 7